



Universidad Veracruzana

Nombre del alumno:

Jorge Samuel Solano Dorantes

Matrículas:

zS22019636

Experiencia Educativa:

Diseño de Aplicaciones Web

Docente Asesor:

Yuliana Berumen Diaz

Actividad:

Documentación Ágora

ÍNDICE

1. Introducción	1
1.1 Descripción General	1
1.2 Objetivos	1
1.3 Alcance	2
1.4 Público Objetivo	2
2. Requisitos	2
2.1 Requisitos Funcionales	2
RF-01 Gestión de Autenticación y Sesión	2
RF-02 Gestión de Perfil de Usuario	3
RF-03 Gestión de Comunidades	3
RF-04 Gestión de Publicaciones	4
RF-05 Gestión de Comentarios	5
RF-06 Sistema de Votos	5
RF-07 Búsqueda	6
RF-08 Panel de Moderación	6
2.2 Requisitos No Funcionales	6
RNF-01 Seguridad	6
RNF-02 Rendimiento	7
RNF-03 Usabilidad	7
RNF-04 Mantenibilidad	7
RNF-05 Portabilidad y Despliegue	8
RNF-06 Compatibilidad	8
3. Diseño	8
3.1 Arquitectura Utilizada	8
3.1.1 Arquitectura General del Sistema	8
3.1.2 Arquitectura del Backend — Capas	9
3.1.3 Arquitectura del Frontend — SPA con React	9
3.2 Diseño de la Base de Datos	9
3.2.1 Decisiones de Modelado NoSQL	10
3.2.2 Colección USUARIO	10
3.2.3 Colección COMUNIDAD	11
3.2.4 Colección PUBLICACION	12
3.2.5 Colección COMENTARIO	13
3.2.6 Colección VOTO	14

3.3 Decisiones de Implementación	14
3.3.1 Cookies HttpOnly para el JWT	14
3.3.2 Actualizaciones Atómicas con MongoTemplate	15
3.3.3 Contadores Desnormalizados	15
3.3.4 Materialized Path para Comentarios Anidados	15
3.3.5 Borrado Lógico de Comentarios	15
3.3.6 Validación de Formatos con JSON Schema en MongoDB Atlas	15
3.3.7 Frontend Servido como Recurso Estático	15
3.4 Diagramas UML	16
3.4.1 Diagrama de Casos de Uso	16
3.4.2 Diagrama de Clases del Backend	18
3.4.3 Diagrama de Secuencia — Inicio de Sesión y Acceso Protegido	19
3.4.4 Diagrama de Secuencia — Publicar y Votar	20
3.4.5 Diagrama de Secuencia — Publicar Comentario Anidado	20
3.4.6 Diagrama de Componentes	21
4. Implementación	22
4.1 Descripción general de la aplicación móvil	22
4.2 Backend - Spring Boot	22
4.2.1 Tecnologías y dependencias principales	22
4.2.2 Estructura de paquetes	23
4.2.3 Controladores REST	23
4.2.4 Capa de seguridad	24
4.2.5 Servicios de negocio	25
4.2.6 Instrucciones NoSQL utilizadas en MongoDB	25
4.3 Frontend - React con Vite	26
4.3.1 Tecnologías y dependencias principales	27
4.3.2 Estructura de directorios	27
4.3.3 Enrutamiento y protección de rutas	28
4.3.4 Contexto de usuario	29
4.3.5 Comunicación con la API	30
4.3.6 Componentes principales	31
4.3.7 Construcción y despliegue	32
5. Resultados	33
5.1 Autenticación y Perfil	33
5.2 Comunidades	35
5.3 Publicaciones	37

5.4 Comentarios y votos	42
5.5 Búsqueda	45
5.6 Panel de moderación	45
6. Conclusiones	47
6.1 Logros	47
6.2 Limitaciones	48
7. Referencias	50

ÍNDICE DE ILUSTRACIONES

Ilustración 1. Diagrama de casos de uso.....	17
Ilustración 2. Diagrama de clases del backend	18
Ilustración 3. Diagrama de secuencia de inicio de sesión y el acceso protegido	19
Ilustración 4. Diagrama de secuencia para publicar y votar.....	20
Ilustración 5. Diagrama de secuencia para publicar un comentario anidado	21
Ilustración 6. Diagrama de componentes	22
Ilustración 7. Pantalla de inicio de sesión.....	33
Ilustración 8. Pantalla de registro	34
Ilustración 9. Pantalla de configuración inicial del perfil.....	34
Ilustración 10. Página de perfil de usuario.....	35
Ilustración 11. Listado de comunidades suscritas	35
Ilustración 12. Listado de comunidades públicas	36
Ilustración 13. Modal de creación de comunidad.....	36
Ilustración 14. Página de una comunidad.....	37
Ilustración 15. Feed de publicaciones (página de inicio)	37
Ilustración 16. Formulario de creación de publicación	38
Ilustración 17. Página de detalle de una publicación de tipo texto.....	39
Ilustración 18. Página de detalle de una publicación de tipo imagen.....	40
Ilustración 19. Página de detalles de una publicación de tipo enlace	41
Ilustración 20. Publicación fijada	41
Ilustración 21. Hilo de comentarios anidados	42
Ilustración 22. Sistema de votos activos en una publicación	43
Ilustración 23. Sistema de votos activos en comentarios	44
Ilustración 24. Página de resultados de una búsqueda completa	45
Ilustración 25. Panel de moderación	45
Ilustración 26. Panel de administración	46
Ilustración 27. Panel de moderación de una publicación.....	46

ÍNDICE DE TABLAS

Tabla 1. Atributos de la colección: Usuario	11
Tabla 2. Atributos de la colección: Comunidad.....	12
Tabla 3. Atributos de la colección: Comunidad.....	13
Tabla 4. Atributos de la colección: Comentario	14
Tabla 5. Atributos de la colección: Voto	14
Tabla 6. Librerías y dependencias (con sus funcionalidades) utilizadas en el backend.....	23
Tabla 7. Librerías y dependencias (con sus funcionalidades) utilizadas en el frontend	27
Tabla 8. Rutas públicas y sus componentes utilizados durante el enrutamiento	28
Tabla 9. Rutas privadas y sus componentes utilizados durante el enrutamiento	29
Tabla 10. Principales endpoints consumidos desde el frontend	31

1. Introducción

1.1 Descripción General

Ágora es una aplicación web de foro comunitario inspirada en la plataforma Reddit, desarrollada como proyecto para la materia de Desarrollo de Aplicaciones Web (DAW). El nombre hace referencia al ágora griega, el espacio público de reunión, debate e intercambio de ideas en las polis de la Antigua Grecia, metáfora que refleja fielmente el propósito del sistema: proporcionar un espacio digital donde las personas puedan crear comunidades, compartir contenido y participar en discusiones organizadas mediante votos y comentarios.

La plataforma permite a los usuarios registrarse, unirse a comunidades de temas específicos, publicar contenido de distintos tipos (texto, imagen o enlace), votar publicaciones y comentarios, así como moderar sus propias comunidades. Se trata de un sistema de red social orientado a la discusión temática, con un mecanismo de reputación basado en karma acumulado a partir de los votos recibidos.

1.2 Objetivos

Objetivo general:

Diseñar e implementar una aplicación web funcional de foro comunitario tipo Reddit, aplicando una arquitectura cliente-servidor con separación de capas, autenticación basada en tokens, y persistencia de datos en una base de datos NoSQL orientada a documentos.

Objetivos específicos:

- Implementar un sistema completo de autenticación y autorización mediante JSON Web Tokens (JWT), garantizando el acceso seguro a los recursos de la API.
- Desarrollar un backend RESTful en Java con Spring Boot que exponga los servicios necesarios para gestionar usuarios, comunidades, publicaciones, comentarios y votos.
- Construir una interfaz de usuario dinámica y reactiva con React y Vite, consumiendo la API del backend mediante peticiones HTTP.
- Modelar y gestionar la persistencia de datos utilizando MongoDB como base de datos NoSQL orientada a documentos, empleando Spring Data MongoDB como capa de abstracción.
- Implementar un sistema de comunidades con roles diferenciados (usuario, moderador y administrador), incluyendo funciones de moderación como fijar, bloquear y eliminar publicaciones.
- Proveer un sistema de votación positiva/negativa que afecte el puntaje de publicaciones y comentarios, así como el karma acumulado de los usuarios.
- Implementar un motor de búsqueda de comunidades y publicaciones dentro de la plataforma.

1.3 Alcance

El alcance del proyecto comprende el desarrollo de una aplicación web de uso interno/académico con las siguientes características y limitaciones:

Dentro del alcance:

La aplicación incluye registro e inicio de sesión de usuarios con configuración de perfil (fotografía de perfil en formato URL, biografía y nombre de usuario). Los usuarios pueden explorar un listado de comunidades públicas, suscribirse a ellas o crear comunidades propias, tanto públicas como privadas. Dentro de cada comunidad es posible realizar publicaciones de tipo texto, enlace e imagen; comentar en publicaciones con soporte de hilos de respuesta anidados; y votar positiva o negativamente tanto publicaciones como comentarios. Los moderadores de una comunidad tienen acceso a un panel de moderación desde donde pueden fijar o bloquear publicaciones, gestionar las reglas de la comunidad e invitar a nuevos moderadores. El sistema contempla también el rol de administrador global con capacidades extendidas.

La búsqueda de contenido está disponible mediante una barra de búsqueda que permite localizar comunidades y publicaciones por nombre o título. El frontend está construido como una SPA (Single Page Application) con rutas protegidas que requieren autenticación activa para ser accedidas.

Fuera del alcance:

El sistema no implementa notificaciones en tiempo real, mensajería directa entre usuarios, sistema de reportes de contenido inapropiado, ni integración con servicios externos de almacenamiento de imágenes (las imágenes se referencian por URL). No incluye soporte para aplicación móvil nativa, ni paginación infinita con carga asíncrona en scroll. El despliegue en producción en un entorno de nube pública no forma parte del alcance del proyecto, siendo suficiente su ejecución en entorno local.

1.4 Público Objetivo

La aplicación está dirigida a cualquier usuario con acceso a un navegador web moderno que desee participar en foros de discusión organizados por comunidades temáticas. En el contexto académico del proyecto, el público objetivo inmediato es el equipo docente evaluador y los propios desarrolladores.

2. Requisitos

2.1 Requisitos Funcionales

Los requisitos funcionales describen las funciones específicas que el sistema debe ser capaz de ejecutar. Se organizan por módulo funcional.

RF-01 Gestión de Autenticación y Sesión

RF-01.1 — Registro de usuario: El sistema debe permitir a un nuevo usuario crear una cuenta proporcionando un nombre de usuario único, un correo electrónico válido y una

contraseña. Todos los campos son obligatorios. El sistema debe verificar que el correo electrónico no esté previamente registrado antes de crear la cuenta.

RF-01.2 — Inicio de sesión: El sistema debe autenticar a un usuario mediante su correo electrónico y contraseña. Ante credenciales válidas, el sistema genera un JSON Web Token (JWT) con duración de 8 horas, que se almacena como una cookie HttpOnly en el navegador del cliente.

RF-01.3 — Cierre de sesión: El sistema debe permitir al usuario cerrar sesión. Al hacerlo, se elimina la cookie que contiene el JWT, invalidando la sesión activa del lado del cliente.

RF-01.4 — Verificación de sesión activa: El sistema debe exponer un endpoint que permita al frontend verificar si el usuario tiene una sesión válida, retornando sus datos básicos (id, nombre de usuario, correo, rol y foto de perfil) extraídos del token JWT.

RF-01.5 — Protección de rutas: Todas las rutas de la aplicación web, a excepción de las páginas de inicio de sesión, registro y configuración inicial de perfil, deben requerir autenticación activa. Si el usuario no está autenticado, el sistema lo redirige a la página de inicio de sesión.

RF-02 Gestión de Perfil de Usuario

RF-02.1 — Configuración inicial de perfil: Después del registro, el sistema debe dirigir al usuario a una página de configuración inicial donde puede establecer su foto de perfil (mediante URL) y una biografía.

RF-02.2 — Visualización de perfil: El sistema debe mostrar una página de perfil pública para cada usuario, accesible mediante su nombre de usuario. Esta página presenta la foto de perfil, nombre de usuario, karma acumulado, biografía y las publicaciones realizadas por ese usuario.

RF-02.3 — Edición de perfil: El sistema debe permitir al usuario autenticado actualizar su foto de perfil y su biografía. Un administrador del sistema también puede modificar el perfil de cualquier usuario.

RF-02.4 — Sistema de karma: El sistema debe mantener un contador de karma por usuario, que se incrementa o decrementa en función de los votos positivos y negativos que reciben sus publicaciones y comentarios.

RF-03 Gestión de Comunidades

RF-03.1 — Listado de comunidades: El sistema debe mostrar una lista de todas las comunidades públicas existentes, incluyendo su nombre, descripción, ícono y número total de miembros.

RF-03.2 — Creación de comunidades: Un usuario autenticado puede crear una nueva comunidad especificando nombre (obligatorio y único), descripción, imagen de banner, ícono, reglas y si la comunidad es pública o privada. El creador de la comunidad queda automáticamente suscrito a ella y es designado como su moderador principal.

RF-03.3 — Suscripción y cancelación de suscripción: El sistema debe permitir a cualquier usuario autenticado suscribirse o cancelar su suscripción a una comunidad pública. El contador de miembros de la comunidad debe actualizarse de forma correspondiente.

RF-03.4 — Edición de comunidad: Los moderadores de una comunidad pueden actualizar su nombre, descripción, banner, ícono y reglas. El sistema verifica que el nuevo nombre no esté en uso por otra comunidad.

RF-03.5 — Eliminación de comunidad: El creador de la comunidad o un administrador del sistema puede eliminar permanentemente una comunidad.

RF-03.6 — Comunidades privadas e invitaciones: El sistema debe soportar comunidades de tipo privado, las cuales no aparecen en el listado público general. Los moderadores pueden invitar usuarios específicos mediante su ID. El usuario invitado recibe una notificación de invitación pendiente y puede aceptarla o rechazarla desde su interfaz.

RF-03.7 — Gestión de moderadores: Los moderadores existentes o un administrador pueden agregar nuevos moderadores a una comunidad. El creador de la comunidad puede remover moderadores, excepto a sí mismo.

RF-03.8 — Expulsión de miembros: Los moderadores y administradores pueden expulsar a miembros de una comunidad, lo que cancela su suscripción y, si era moderador, también elimina ese rol. No es posible expulsar al creador de la comunidad.

RF-04 Gestión de Publicaciones

RF-04.1 — Creación de publicaciones: Un usuario autenticado puede crear publicaciones dentro de una comunidad existente. Cada publicación debe tener un título (obligatorio) y al menos un tipo de contenido: texto (contenido en formato libre), enlace (URL externa) o imagen (URL de imagen). Adicionalmente, se puede asignar una etiqueta de categoría a la publicación.

RF-04.2 — Listado y paginación de publicaciones: El sistema debe listar publicaciones con soporte de paginación, permitiendo filtrar por comunidad o por autor, y ordenarlas por fecha de creación (más recientes primero) o por puntaje de votos (más populares primero). El máximo de publicaciones por petición es de 100.

RF-04.3 — Visualización individual de publicación: El sistema debe mostrar una página detallada de cada publicación con su contenido completo, datos del autor, puntaje de votos, total de comentarios, etiqueta y estado (fijada o bloqueada).

RF-04.4 — Edición de publicación: El autor de una publicación o un administrador del sistema puede modificar el título, el contenido textual y la etiqueta de esta.

RF-04.5 — Eliminación de publicación: El autor de la publicación, un moderador de la comunidad donde se publicó, o un administrador del sistema puede eliminar una publicación de forma permanente.

RF-04.6 — Fijar publicación: Los moderadores y administradores pueden fijar o desfijar publicaciones dentro de una comunidad. Las publicaciones fijadas se destacan visualmente en la interfaz.

RF-04.7 — Bloquear comentarios: Los moderadores y administradores pueden bloquear o desbloquear la sección de comentarios de una publicación, impidiendo que se agreguen nuevos comentarios mientras permanezca bloqueada.

RF-05 Gestión de Comentarios

RF-05.1 — Creación de comentarios: Un usuario autenticado puede comentar en cualquier publicación que no tenga sus comentarios bloqueados. Los comentarios requieren que se especifique el contenido (texto no vacío) y el ID de la publicación.

RF-05.2 — Respuestas anidadas (hilos): El sistema debe soportar comentarios como respuesta a otros comentarios, formando hilos de conversación. Cada comentario puede tener un comentario padre, y el sistema almacena el camino jerárquico (hilo) para reconstruir la estructura del árbol en el frontend.

RF-05.3 — Listado de comentarios: El sistema devuelve todos los comentarios asociados a una publicación, organizados de forma que permita al frontend construir la vista de árbol.

RF-05.4 — Edición de comentario: El autor del comentario puede modificar su contenido. Un administrador también puede editar cualquier comentario.

RF-05.5 — Eliminación lógica de comentario: Un comentario puede ser eliminado por su autor, por un moderador de la comunidad o por un administrador. La eliminación es lógica: el comentario permanece en la base de datos con el campo **eliminado = true**, y su contenido se reemplaza en la respuesta por el texto **[comentario eliminado]**, preservando la estructura del hilo.

RF-06 Sistema de Votos

RF-06.1 — Votar publicaciones y comentarios: Un usuario autenticado puede emitir un voto positivo (+1) o negativo (-1) sobre cualquier publicación o comentario. El tipo de objetivo (publicación o comentario) se especifica en la petición.

RF-06.2 — Cambio y retiro de voto: Si el usuario ya votó un mismo elemento con el mismo valor, el sistema elimina el voto (retiro). Si vota con un valor diferente al voto existente, el sistema actualiza el voto al nuevo valor. De este modo, un usuario no puede emitir más de un voto activo sobre el mismo elemento.

RF-06.3 — Actualización de puntaje: Al procesar un voto, el sistema actualiza el contador **puntaje_votos** del elemento votado, así como los contadores específicos **votos_positivos** y **votos_negativos** en las publicaciones.

RF-06.4 — Actualización de karma: Cuando se vota una publicación o comentario, el sistema actualiza el karma del usuario autor del contenido votado, sumando o restando según el valor del voto.

RF-06.5 — Consulta de voto activo: El sistema permite consultar si el usuario autenticado ya ha votado un elemento específico y cuál es el valor de ese voto, para que el frontend pueda reflejar el estado del voto en la interfaz gráfica.

RF-07 Búsqueda

RF-07.1 — Búsqueda global: El sistema debe ofrecer un endpoint de búsqueda que, dado un término de consulta, retorne resultados agrupados en tres categorías: comunidades públicas cuyo nombre o descripción contenga el término, publicaciones cuyo título contenga el término (sin distinción de mayúsculas y minúsculas), y usuarios cuyo nombre de usuario contenga el término.

RF-07.2 — Interfaz de búsqueda: El frontend debe contar con una barra de búsqueda accesible desde el layout principal, que muestre sugerencias en tiempo real mientras el usuario escribe y permita navegar a una página de resultados completa.

RF-08 Panel de Moderación

RF-08.1 — Acceso al panel: Los moderadores de una comunidad y los administradores del sistema pueden acceder a un panel de moderación específico para cada comunidad.

RF-08.2 — Gestión de publicaciones desde el panel: El panel de moderación permite ver, fijar, desfijar, bloquear, desbloquear y eliminar publicaciones de la comunidad.

RF-08.3 — Gestión de miembros desde el panel: El panel permite visualizar la lista de miembros de la comunidad y expulsarlos si se cuenta con los permisos necesarios.

RF-08.4 — Gestión de moderadores desde el panel: Desde el panel, el creador de la comunidad puede agregar y remover moderadores. Los moderadores que no son el creador pueden agregar otros moderadores, pero no removerlos.

RF-08.5 — Edición de información de la comunidad desde el panel: El panel permite actualizar el nombre, descripción, banner, ícono y reglas de la comunidad.

2.2 Requisitos No Funcionales

Los requisitos no funcionales definen las restricciones de calidad y comportamiento del sistema, independientemente de las funciones específicas que realiza.

RNF-01 Seguridad

RNF-01.1 — Autenticación basada en tokens: El sistema utiliza JWT firmados con HMAC-SHA para autenticar cada petición al backend. Los tokens se transmiten mediante cookies HttpOnly, lo que impide su acceso desde JavaScript del lado del cliente y mitiga ataques XSS.

RNF-01.2 — Cifrado de contraseñas: Las contraseñas de los usuarios nunca se almacenan en texto plano. El sistema utiliza el algoritmo bcrypt (biblioteca jBCrypt) para el hashing de contraseñas antes de persistirlas en la base de datos.

RNF-01.3 — Control de acceso basado en roles: El sistema implementa tres niveles de rol: **usuario** (acceso estándar), **moderador** (gestión dentro de su comunidad) y **admin** (acceso total al sistema). Las operaciones sensibles validan el rol del solicitante antes de ejecutarse.

RNF-01.4 — Protección CORS: El backend configura una política CORS que permite credenciales, controla los orígenes autorizados y los métodos HTTP permitidos (GET, POST, PUT, DELETE, OPTIONS, PATCH).

RNF-01.5 — Deshabilitación de CSRF: Dado que la autenticación se basa en cookies gestionadas por el servidor y el sistema está diseñado para ser consumido exclusivamente por el frontend propio, la protección CSRF de Spring Security se deshabilita explícitamente para no interferir con las peticiones de la SPA.

RNF-02 Rendimiento

RNF-02.1 — Límite de resultados por petición: Las peticiones de listado de publicaciones tienen un límite máximo de 100 registros por consulta, previniendo sobrecargas en la base de datos y en el tiempo de respuesta.

RNF-02.2 — Paginación del servidor: El sistema soporta paginación del lado del servidor para publicaciones, mediante los parámetros **pagina** y **limite**, lo que reduce el volumen de datos transferidos por petición.

RNF-02.3 — Respuesta ágil de la SPA: Al ser una Single Page Application, la navegación entre páginas no genera recargas completas del navegador, lo que resulta en una experiencia de usuario más fluida y tiempos de transición percibidos más cortos.

RNF-03 Usabilidad

RNF-03.1 — Interfaz intuitiva: La interfaz de usuario sigue convenciones visuales familiares de foros y redes sociales, con elementos como tarjetas de publicación, sistema de votación con íconos de flechas arriba/abajo, avatares de usuario y secciones claramente diferenciadas.

RNF-03.2 — Retroalimentación visual: El sistema provee retroalimentación inmediata al usuario ante sus acciones: mensajes de error en formularios, estados de carga, confirmación de operaciones exitosas y actualización instantánea de contadores de votos.

RNF-03.3 — Rutas protegidas con redirección: Las rutas que requieren autenticación redirigen automáticamente al usuario al login si no tiene sesión activa, y las rutas públicas redirigen al inicio si el usuario ya está autenticado, evitando estados inconsistentes en la navegación.

RNF-04 Mantenibilidad

RNF-04.1 — Arquitectura por capas: El backend sigue una arquitectura en capas con separación clara entre controladores (capa de presentación/API), servicios (lógica de negocio) y repositorios (acceso a datos), lo que facilita el mantenimiento y la extensión del sistema.

RNF-04.2 — Separación frontend-backend: El frontend y el backend son proyectos completamente independientes que se comunican exclusivamente a través de la API REST, permitiendo que cada capa pueda desarrollarse, probarse y desplegarse de forma separada.

RNF-04.3 — Componetización del frontend: La interfaz de usuario está construida con componentes React reutilizables (tarjetas de publicación, árbol de comentarios, componente de votos, barra de búsqueda, etc.), facilitando la modificación y extensión de la UI.

RNF-05 Portabilidad y Despliegue

RNF-05.1 — Configuración por variables de entorno: Los parámetros sensibles del sistema (URI de MongoDB y secreto JWT) se configuran mediante variables de entorno (**MONGODB_URI** y **JWT_SECRET**), lo que facilita el despliegue en distintos entornos sin modificar el código fuente.

RNF-05.2 — Servidor embebido: El backend usa el servidor embebido de Spring Boot (Tomcat), lo que elimina la necesidad de un servidor de aplicaciones externo y simplifica el despliegue. El sistema escucha en el puerto 9090 y en todas las interfaces de red (**0.0.0.0**).

RNF-05.3 — Frontend servido desde el backend: La distribución del frontend (generada con **vite build**) se sirve directamente desde el backend Spring Boot como recursos estáticos, permitiendo desplegar la aplicación completa como un único artefacto JAR.

RNF-06 Compatibilidad

RNF-06.1 — Compatibilidad con navegadores modernos: La aplicación está desarrollada con React 19 y Vite 8, y su funcionamiento es compatible con las versiones actuales de los principales navegadores web (Chrome, Firefox, Edge, Safari).

RNF-06.2 — API RESTful estándar: El backend expone una API RESTful que utiliza los verbos HTTP estándar (GET, POST, PUT, DELETE, PATCH) y retorna respuestas en formato JSON, garantizando interoperabilidad con cualquier cliente HTTP.

3. Diseño

3.1 Arquitectura Utilizada

Ágora sigue una arquitectura **cliente-servidor de dos capas**, donde el frontend y el backend son proyectos independientes que se comunican exclusivamente a través de una **API REST sobre HTTP/HTTPS**. Esta separación permite que cada capa pueda desarrollarse, probarse y mantenerse de forma autónoma.

3.1.1 Arquitectura General del Sistema

La aplicación se compone de tres elementos principales: el cliente (navegador web), el servidor de aplicaciones (backend Spring Boot) y el servidor de base de datos (MongoDB Atlas). El flujo de comunicación es siempre iniciado por el cliente hacia el backend, y el backend es el único componente que interactúa directamente con la base de datos.

En producción, el frontend compilado (distribución generada por Vite) se sirve como recursos estáticos directamente desde el backend Spring Boot, lo que permite empaquetar y desplegar toda la aplicación como un único artefacto JAR. Durante el desarrollo, el frontend corre en un servidor de desarrollo Vite independiente y consume la API del backend mediante peticiones HTTP con CORS habilitado.

3.1.2 Arquitectura del Backend — Capas

El backend implementa una arquitectura en **tres capas** bien definidas:

La **capa de presentación (Controladores)** es responsable de recibir las peticiones HTTP, extraer parámetros y delegar el trabajo a la capa de servicio o repositorio. Cada controlador agrupa los endpoints de un recurso específico: **ControladorAutenticacion**, **ControladorUsuario**, **ControladorComunidad**, **ControladorPublicacion**, **ControladorComentario**, **ControladorVoto** y **ControladorBusqueda**.

La **capa de negocio (Servicios)** encapsula la lógica de negocio más compleja que involucra múltiples entidades o reglas de negocio no triviales. Los servicios implementados son **ServicioAutenticacion** (registro, login, hashing con bcrypt y generación de JWT), **ServicioComentario** (construcción del árbol de comentarios con *materialized path* e incremento de contadores) y **ServicioVoto** (procesamiento de votos con lógica de retiro y cambio, actualización de puntajes y karma).

La **capa de acceso a datos (Repositorios)** abstrae las operaciones de consulta y persistencia sobre MongoDB mediante interfaces que extienden **MongoRepository** de Spring Data MongoDB. Para operaciones más complejas que requieren actualizaciones atómicas o queries dinámicas, los controladores y servicios utilizan directamente **MongoTemplate**.

3.1.3 Arquitectura del Frontend — SPA con React

El frontend es una **Single Page Application (SPA)** construida con React 19 y Vite 8. La navegación se gestiona completamente en el cliente mediante React Router DOM 7, sin recarga de página entre rutas. La arquitectura interna del frontend se organiza en:

Páginas, que representan las vistas principales de la aplicación: **Login**, **Registro**, **PerfilSetup**, **Inicio**, **Comunidades**, **PaginaComunidad**, **PaginaPublicacion**, **PaginaCrearPublicacion**, **PaginaPerfil** y **PaginaBusqueda**.

Componentes reutilizables, que encapsulan piezas de UI independientes: **TarjetaPublicacion**, **ArbolComentarios**, **ComponenteVotos**, **BarraBusqueda**, **ModalCrearComunidad**, **PanelModeracion**, **LayoutPrincipal**, **RutaProtegida** y **RutaPublica**.

Contexto de usuario (ContextoUsuario), que provee el estado de autenticación globalmente a toda la aplicación mediante el Context API de React, evitando el paso de props entre componentes.

3.2 Diseño de la Base de Datos

La base de datos utilizada es **MongoDB**, un sistema de gestión de bases de datos NoSQL orientado a documentos. Los datos se almacenan en formato BSON (Binary JSON) dentro de cinco colecciones principales, validadas mediante JSON Schema a nivel de la base de datos en MongoDB Atlas.

3.2.1 Decisiones de Modelado NoSQL

A diferencia de un modelo relacional, MongoDB favorece el **embedding** (incrustar documentos dentro de otros) para datos que siempre se consultan juntos, y las **referencias por ObjectId** para datos que se consultan de forma independiente o que pueden crecer indefinidamente.

En Ágora se aplicaron las siguientes decisiones de modelado:

Las **reglas de una comunidad** se modelaron como un array embebido dentro del documento de **COMUNIDAD**, ya que son datos que pertenecen exclusivamente a esa comunidad, se consultan siempre en conjunto con ella y no tienen identidad propia fuera de ese contexto.

Los **moderadores y miembros** se almacenan como arrays de **ObjectId** que referencian documentos de la colección **USUARIO**, ya que un usuario existe de forma independiente y puede pertenecer a múltiples comunidades.

Los **comentarios** se almacenan en una colección separada **COMENTARIO** en lugar de estar embebidos en la publicación, porque su número es ilimitado y el patrón *materialized path* (campo **hilo**) permite reconstruir la jerarquía sin necesidad de joins recursivos.

Los **votos** se almacenan en una colección separada **VOTO** en lugar de en el documento votado, lo que permite consultas eficientes del tipo "¿ya votó este usuario este elemento?" y evita que los documentos de publicaciones o comentarios crezcan de forma ilimitada.

3.2.2 Colección USUARIO

Almacena la información de cada usuario registrado en la plataforma. El campo **nombre_usuario** identifica al usuario públicamente, mientras que **email** es el identificador usado para autenticación. La contraseña se almacena cifrada con bcrypt. El campo **karma** acumula los votos positivos y negativos recibidos en publicaciones y comentarios. El campo **comunidades_suscritas** es un array de referencias (**ObjectId**) a las comunidades a las que el usuario se ha suscrito. El campo **rol** solo admite los valores "**usuario**" y "**admin**", ya que el rol de moderador es específico de cada comunidad y se gestiona dentro del documento de **COMUNIDAD**.

Campo	Tipo BSON	Requerido	Descripción
_id	ObjectId	Auto	Identificador único generado por MongoDB
nombre_usuario	string	Sí	Nombre público del usuario

email	string	Sí	Correo electrónico, validado con regex
contrasena	string	Sí	Hash bcrypt de la contraseña
foto_perfil	string	No	URL de la imagen de perfil
biografia	string	No	Texto libre, máximo 300 caracteres
karma	number	No	Puntos de reputación acumulados
comunidades_suscritas	array<ObjectId>	No	Referencias a comunidades suscritas
creado_en	date	Sí	Fecha de registro
rol	string	No	"usuario" o "admin"

Tabla 1. Atributos de la colección: Usuario

3.2.3 Colección COMUNIDAD

Almacena cada comunidad creada en la plataforma. Las **reglas** se modelan como un array de subdocumentos embebidos con **título** (obligatorio) y **descripcion** opcionales. Los **moderadores** son un array de referencias a usuarios con permiso de gestión sobre la comunidad. El campo **es_privada** controla la visibilidad pública. El campo **invitados_pendientes** es un array de referencias a usuarios con invitación pendiente para comunidades privadas.

Campo	Tipo BSON	Requerido	Descripción
_id	ObjectId	Auto	Identificador único
nombre	string	Sí	Nombre único de la comunidad
descripcion	string	No	Descripción, máximo 500 caracteres
banner	string	No	URL de la imagen de banner
icono	string	No	URL del ícono de la comunidad
reglas	array<objeto>	No	Array embebido de reglas (título + descripción)

moderadores	array<ObjectId>	No	Referencias a usuarios moderadores
total_miembros	int	No	Contador desnormalizado de suscriptores
creado_por	ObjectId	Sí	Referencia al usuario creador
creado_en	date	Sí	Fecha de creación
es_privada	bool	No	Indica si la comunidad es privada
invitados_pendientes	array<ObjectId>	No	Usuarios con invitación pendiente

Tabla 2. Atributos de la colección: Comunidad

3.2.4 Colección PUBLICACION

Almacena cada publicación creada dentro de una comunidad. El campo **tipos** es un array que puede contener uno o más de los valores **"texto"**, **"enlace"** o **"imagen"**, determinando qué campos de contenido adicionales son relevantes. Los campos **puntaje_votos**, **votos_positivos**, **votos_negativos** y **total_comentarios** son **contadores desnormalizados** que se actualizan atómicamente en cada operación de voto o comentario, evitando operaciones costosas de agregación en cada consulta.

Campo	Tipo BSON	Requerido	Descripción
_id	ObjectId	Auto	Identificador único
titulo	string	Sí	Título, entre 3 y 300 caracteres
tipos	array<string>	Sí	Al menos uno: "texto", "enlace", "imagen"
contenido	string	No	Texto de la publicación
enlace	string	No	URL externa
url_imagen	string	No	URL de la imagen
autor	ObjectId	Sí	Referencia al usuario autor
comunidad	ObjectId	Sí	Referencia a la comunidad

puntaje_votos	number	No	Suma neta de votos (+1 y -1)
votos_positivos	int	No	Contador de votos positivos (≥ 0)
votos_negativos	int	No	Contador de votos negativos (≥ 0)
total_comentarios	int	No	Contador de comentarios (≥ 0)
etiqueta	string	No	Etiqueta de categoría opcional
fijada	bool	No	Publicación fijada por moderador
bloqueada	bool	No	Comentarios bloqueados por moderador
creado_en	date	Sí	Fecha de creación
actualizado_en	date	No	Fecha de última modificación

Tabla 3. Atributos de la colección: Comunidad

3.2.5 Colección COMENTARIO

Almacena cada comentario o respuesta dentro de una publicación. La jerarquía de comentarios anidados se implementa mediante el patrón **Materialized Path**: el campo **hilo** es un array de **ObjectId** que contiene la ruta completa desde el comentario raíz hasta el padre inmediato del comentario actual. Si el comentario es raíz (sin padre), **hilo** es un array vacío. El campo **padre_id** indica el padre directo, o **null** si es un comentario raíz. La profundidad máxima permitida es de 10 niveles (maxItems: 10 en el hilo). La eliminación es **lógica**: el campo **eliminado** se pone en **true** y el contenido se oculta en la respuesta de la API, pero el documento se preserva para mantener la estructura del árbol.

Campo	Tipo BSON	Requerido	Descripción
_id	ObjectId	Auto	Identificador único
publicacion_id	ObjectId	Sí	Referencia a la publicación
autor	ObjectId	Sí	Referencia al usuario autor
contenido	string	Sí	Texto del comentario (1–10.000 chars)

padre_id	ObjectId / null	No	Padre directo (null si es raíz)
hilo	array<ObjectId>	Sí	Materialized path desde raíz hasta padre
puntaje_votos	int	No	Puntaje neto de votos
eliminado	bool	No	Borrado suave; preserva estructura del árbol
creado_en	date	Sí	Fecha de creación
actualizado_en	date	No	Fecha de última modificación

Tabla 4. Atributos de la colección: Comentario

3.2.6 Colección VOTO

Registra cada voto emitido por un usuario sobre una publicación o comentario. La combinación **usuario_id + objetivo_id** es funcionalmente única: no puede existir más de un voto activo del mismo usuario sobre el mismo elemento. El campo **tipo_objetivo** indica si el elemento votado es una "**publicacion**" o un "**comentario**". El campo **autor_objetivo_id** almacena una referencia al autor del contenido votado para poder actualizar su karma directamente sin hacer una consulta adicional. El campo **valor** solo admite 1 (positivo) o -1 (negativo).

Campo	Tipo BSON	Requerido	Descripción
_id	ObjectId	Auto	Identificador único
usuario_id	ObjectId	Sí	Usuario que emite el voto
objetivo_id	ObjectId	Sí	Publicación o comentario votado
tipo_objetivo	string	Sí	"publicacion" o "comentario"
autor_objetivo_id	ObjectId	Sí	Autor del contenido votado
valor	number	Sí	1 (positivo) o -1 (negativo)

Tabla 5. Atributos de la colección: Voto

3.3 Decisiones de Implementación

3.3.1 Cookies HttpOnly para el JWT

Se optó por almacenar el JWT en una cookie `HttpOnly` en lugar de en **localStorage** o **sessionStorage** del navegador. Esta decisión impide que scripts de JavaScript accedan al token, reduciendo la superficie de ataque ante vulnerabilidades XSS. La cookie se configura con **setHttpOnly(true)**, **setPath("/")** y una duración de 8 horas, coherente con la expiración del token.

3.3.2 Actualizaciones Atómicas con MongoTemplate

Para operaciones que requieren incrementar o decrementar contadores (karma, puntaje de votos, total de miembros, total de comentarios), se utiliza **MongoTemplate** con operaciones **\$inc** de MongoDB, en lugar de hacer una lectura, modificar en memoria y volver a guardar. Esto garantiza que las actualizaciones sean atómicas y evita condiciones de carrera en escenarios de concurrencia.

3.3.3 Contadores Desnormalizados

Los campos **puntaje_votos**, **votos_positivos**, **votos_negativos** y **total_comentarios** en **PUBLICACION**, y **puntaje_votos** en **COMENTARIO**, son contadores desnormalizados. En una base de datos NoSQL no existen joins, por lo que calcular estos valores mediante agregaciones en cada consulta sería costoso. Al mantenerlos actualizados atómicamente con **\$inc**, las lecturas de publicaciones son simples y eficientes.

3.3.4 Materialized Path para Comentarios Anidados

Para el árbol de comentarios se implementó el patrón **Materialized Path**: cada comentario almacena en su campo **hilo** el array de IDs de todos sus ancestros en orden. Esto permite recuperar todos los comentarios de una publicación en una sola consulta y reconstruir el árbol completo en el frontend ordenando por la longitud y contenido del hilo, sin necesidad de múltiples consultas recursivas. La profundidad máxima está limitada a 10 niveles.

3.3.5 Borrado Lógico de Comentarios

Cuando un comentario es eliminado, no se borra físicamente de la base de datos. En su lugar, se establece el campo **eliminado = true**. Esto preserva la estructura del árbol de respuestas: si un comentario con respuestas fuera eliminado físicamente, sus respuestas quedarían huérfanas y la jerarquía se rompería. La API retorna **[comentario eliminado]** como contenido y **null** como autor para estos casos.

3.3.6 Validación de Formatos con JSON Schema en MongoDB Atlas

Las cinco colecciones tienen validadores de esquema JSON configurados en MongoDB Atlas que aplican restricciones de tipo, longitud, formato (el email mediante expresión regular) y valores permitidos (enums para **rol**, **tipos**, **tipo_objetivo** y **valor**). Esto agrega una capa de integridad de datos a nivel de base de datos, complementaria a las validaciones del backend.

3.3.7 Frontend Servido como Recurso Estático

El resultado de **vite build** se copia al directorio **src/main/resources/dist/** del proyecto Spring Boot, y el controlador **ControladorVistas** sirve el **index.html** para todas las rutas no

reconocidas como API. Esto permite que React Router maneje la navegación del lado del cliente sin que el servidor devuelva 404 al acceder directamente a una URL de la SPA.

3.4 Diagramas UML

Esta sección presenta los diagramas UML correspondientes a la fase de diseño de Ágora.

3.4.1 Diagrama de Casos de Uso

El diagrama de casos de uso representa las interacciones entre los actores del sistema y las funcionalidades que el sistema ofrece. Se identifican tres actores principales:

Actores:

- **Usuario Anónimo:** Visitante sin sesión activa. Solo puede acceder a las páginas de registro e inicio de sesión.
- **Usuario Autenticado:** Usuario con sesión activa. Tiene acceso a la mayoría de las funcionalidades de la plataforma.
- **Moderador:** Hereda todas las capacidades del Usuario Autenticado y adicionalmente gestiona su comunidad.
- **Administrador:** Actor con acceso total al sistema, que hereda las capacidades del Moderador y puede gestionar cualquier comunidad o usuario.

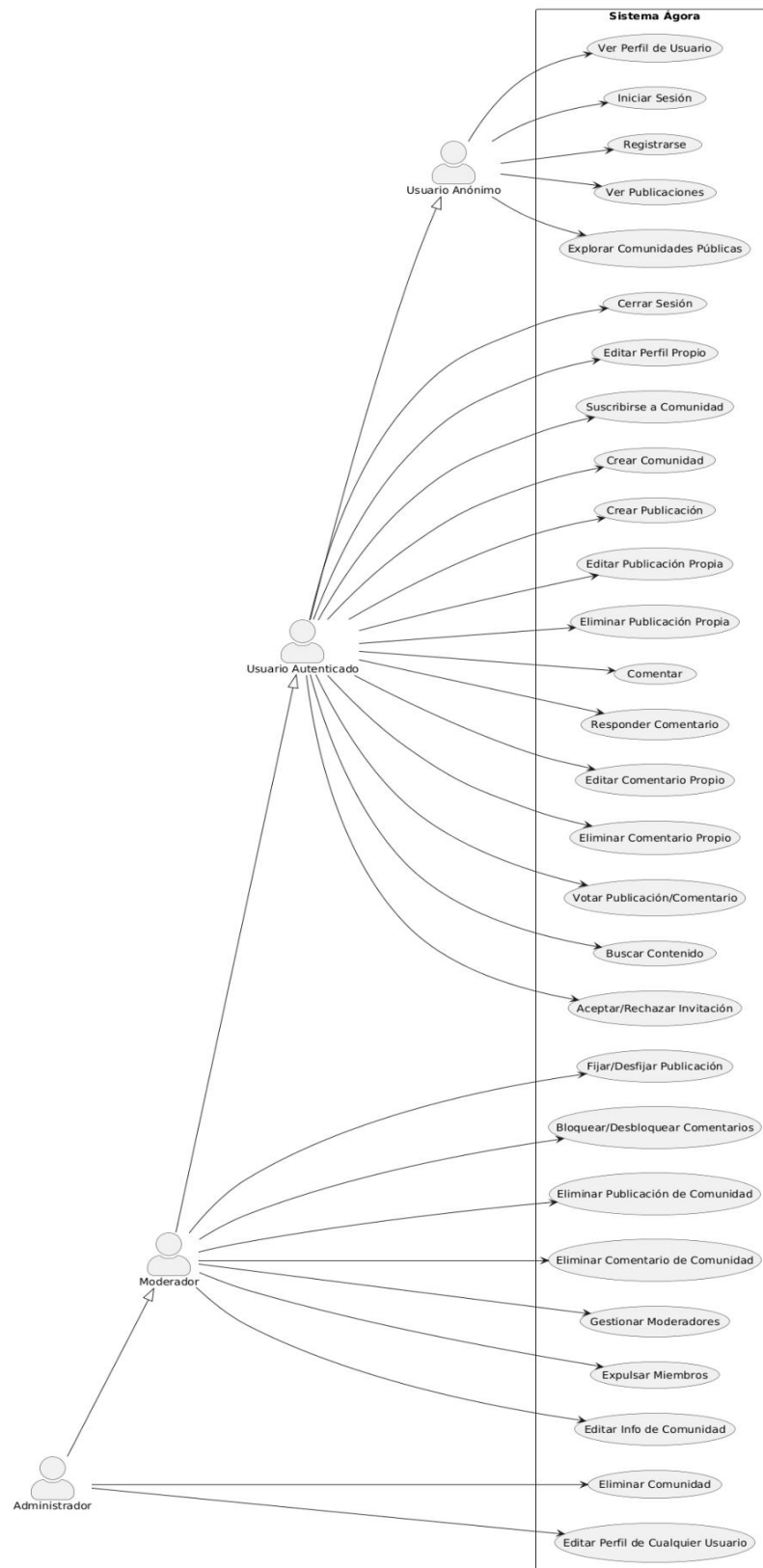


Ilustración 1. Diagrama de casos de uso

Descripción narrativa: El usuario anónimo solo puede registrarse, iniciar sesión, ver perfiles públicos, explorar comunidades y ver publicaciones. Una vez autenticado, puede gestionar su propio perfil, interactuar con comunidades (suscribirse, crear nuevas), publicar,

comentar, responder, votar y buscar contenido. Si posee el rol de moderador en una comunidad, adicionalmente puede fijar publicaciones, bloquear comentarios, eliminar contenido de su comunidad, gestionar moderadores y expulsar miembros. El administrador tiene acceso irrestricto, incluyendo eliminar cualquier comunidad y editar el perfil de cualquier usuario.

3.4.2 Diagrama de Clases del Backend

El diagrama de clases modela las entidades principales del backend de Ágora, sus atributos y las relaciones que existen entre ellas. Las clases representan los documentos MongoDB mapeados mediante Spring Data MongoDB con la anotación **@Document**.

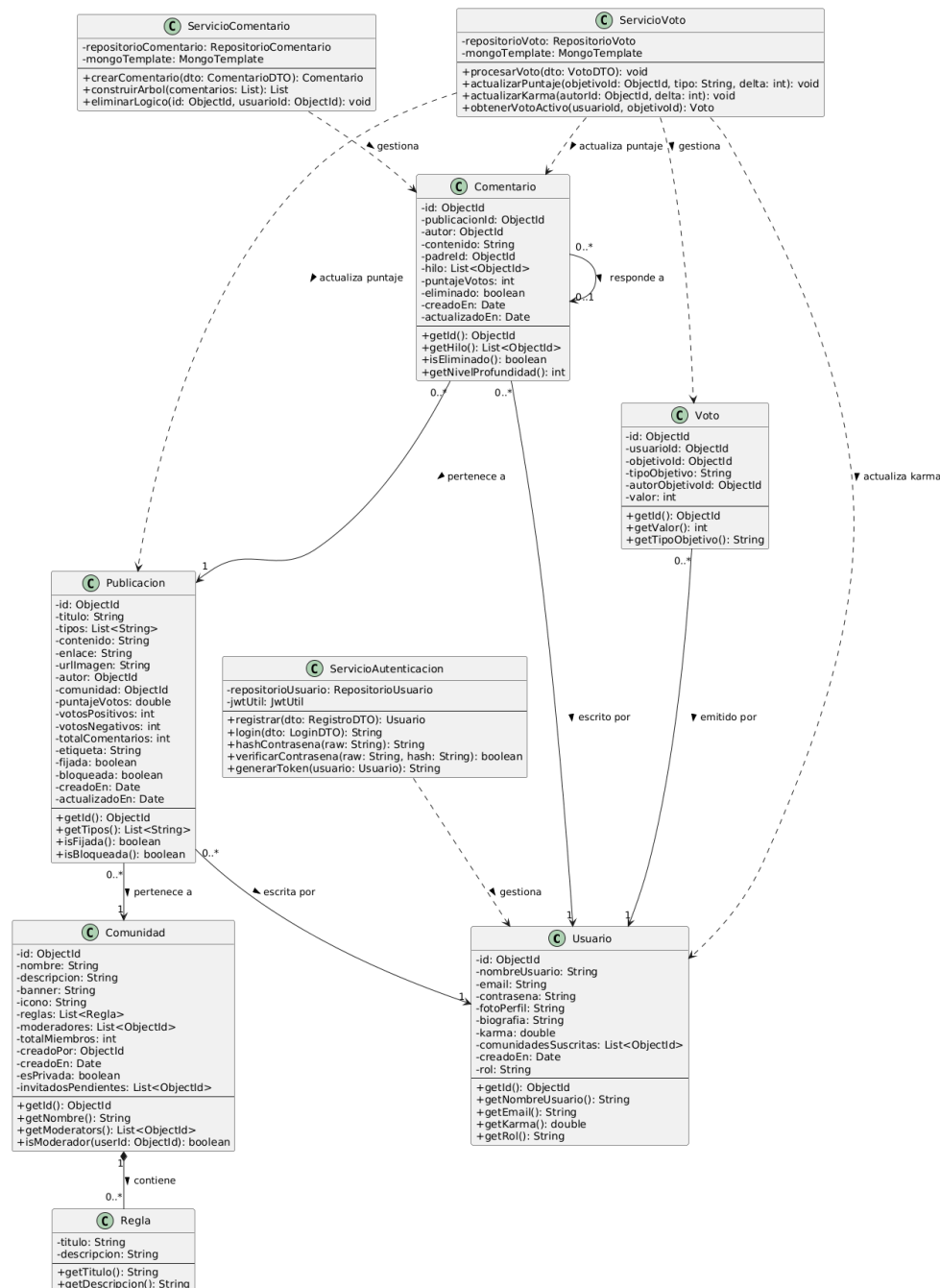


Ilustración 2. Diagrama de clases del backend

Descripción narrativa: La clase **Usuario** es central en el sistema: es referenciada por **Publicacion**, **Comentario** y **Voto** a través de sus ObjectId. La clase **Comunidad** contiene de forma embebida una lista de objetos **Regla** (relación de composición), mientras que sus moderadores y miembros se almacenan como arrays de referencias a **Usuario**. **Publicacion** referencia a su **autor** y a su **comunidad**. **Comentario** se autoreferencia para representar la jerarquía padre-hijo, y almacena el **hilo** como materialized path. **Voto** referencia al usuario votante, al objetivo (publicación o comentario) y al autor del objetivo. Los tres servicios de negocio (**ServicioAutenticacion**, **ServicioComentario**, **ServicioVoto**) interactúan con las entidades correspondientes para encapsular la lógica más compleja.

3.4.3 Diagrama de Secuencia — Inicio de Sesión y Acceso Protegido

Este diagrama ilustra el flujo completo desde que el usuario introduce sus credenciales hasta que accede a un recurso protegido en una petición subsiguiente.

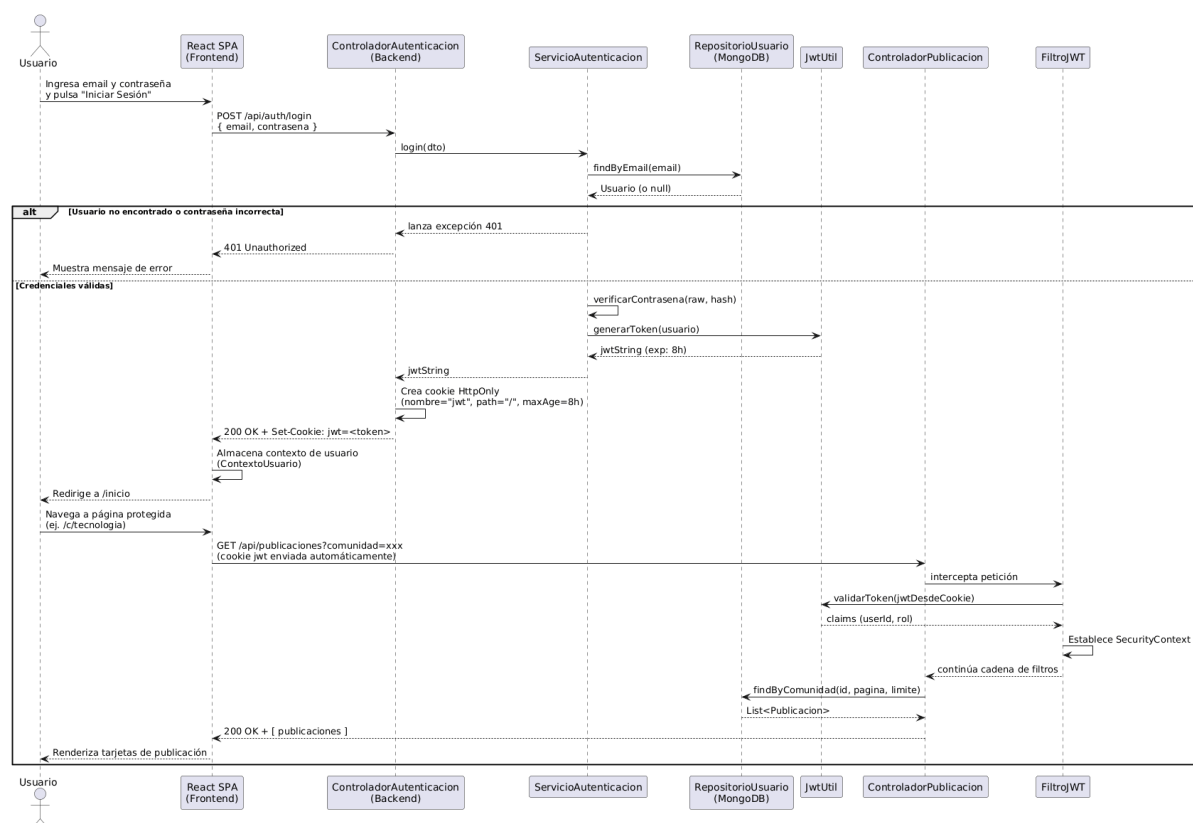


Ilustración 3. Diagrama de secuencia de inicio de sesión y el acceso protegido

Descripción narrativa: El usuario envía sus credenciales al controlador de autenticación. El servicio recupera el usuario de MongoDB, verifica la contraseña con bcrypt y, si es válida, genera un JWT con **JwtUtil**. El backend construye una cookie HttpOnly que contiene el token y la incluye en la respuesta. El frontend almacena los datos del usuario en el **ContextoUsuario** y redirige a la página principal. En peticiones posteriores, el navegador envía automáticamente la cookie. El **FiltroJWT** intercepta cada petición, extrae y valida el token, y establece el contexto de seguridad de Spring Security antes de que la petición llegue al controlador correspondiente.

3.4.4 Diagrama de Secuencia — Publicar y Votar

Este diagrama muestra el flujo de creación de una publicación y el posterior procesamiento de un voto sobre ella, ilustrando la lógica de actualización atómica de contadores.

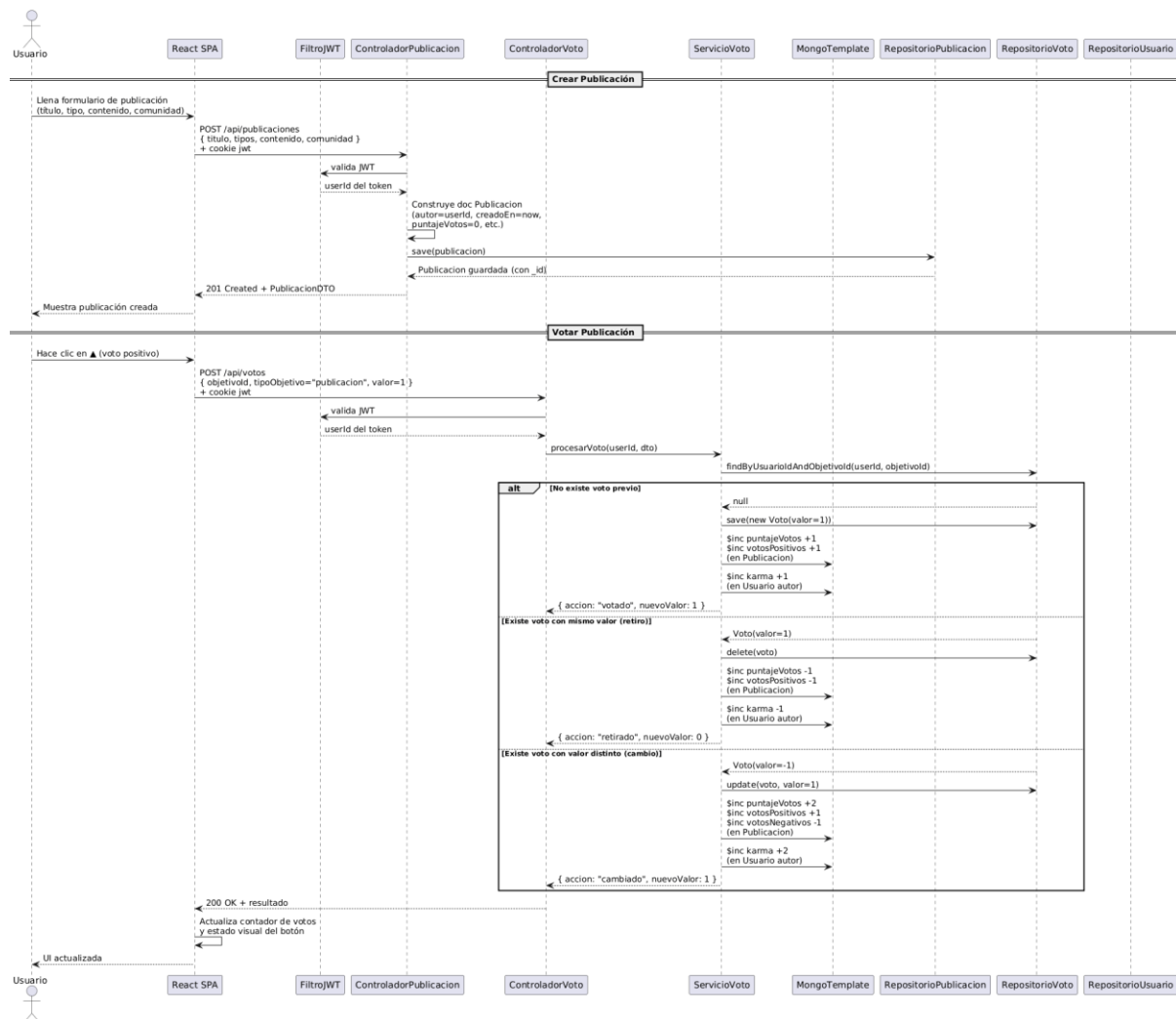


Ilustración 4. Diagrama de secuencia para publicar y votar

Descripción narrativa: Al crear una publicación, el controlador extrae el **userId** del JWT validado por el filtro, construye el documento con valores iniciales en cero para contadores y lo persiste en MongoDB. Para el voto, el **ServicioVoto** consulta primero si existe un voto previo del mismo usuario sobre el mismo objetivo. Si no existe, inserta el voto y aplica **\$inc** atómico sobre **puntajeVotos** y **karma**. Si existe con el mismo valor, lo elimina y revierte los contadores (retiro de voto). Si existe con valor distinto, actualiza el documento y aplica el delta correspondiente (+2 en caso de cambio de -1 a +1). Todas las actualizaciones de contadores se realizan con **MongoTemplate** para garantizar atomicidad.

3.4.5 Diagrama de Secuencia — Publicar Comentario Anidado

Este diagrama muestra el flujo de creación de un comentario en respuesta a otro comentario, ilustrando el funcionamiento del patrón Materialized Path.

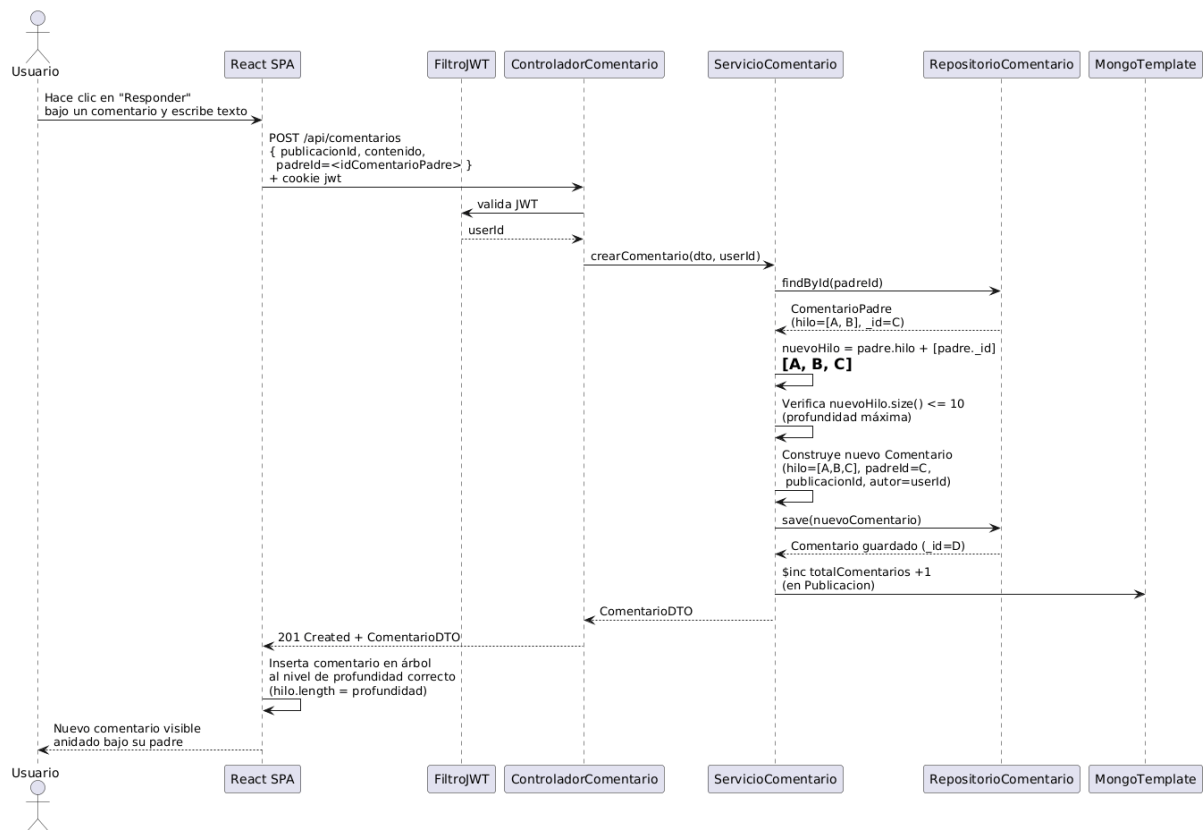


Ilustración 5. Diagrama de secuencia para publicar un comentario anidado

Descripción narrativa: Al crear un comentario en respuesta a otro, el servicio recupera el comentario padre y construye el nuevo **hilo** concatenando el **hilo** del padre con el **_id** del padre. Esto establece el camino completo desde la raíz hasta el nuevo comentario. El servicio verifica que la profundidad no supere 10 niveles antes de persistir. Adicionalmente, incrementa el contador **totalComentarios** de la publicación de forma atómica. En el frontend, la longitud del array **hilo** determina el nivel de indentación al renderizar el árbol de comentarios.

3.4.6 Diagrama de Componentes

El diagrama de componentes representa la estructura de alto nivel del sistema Ágora, mostrando los módulos software principales y sus dependencias.

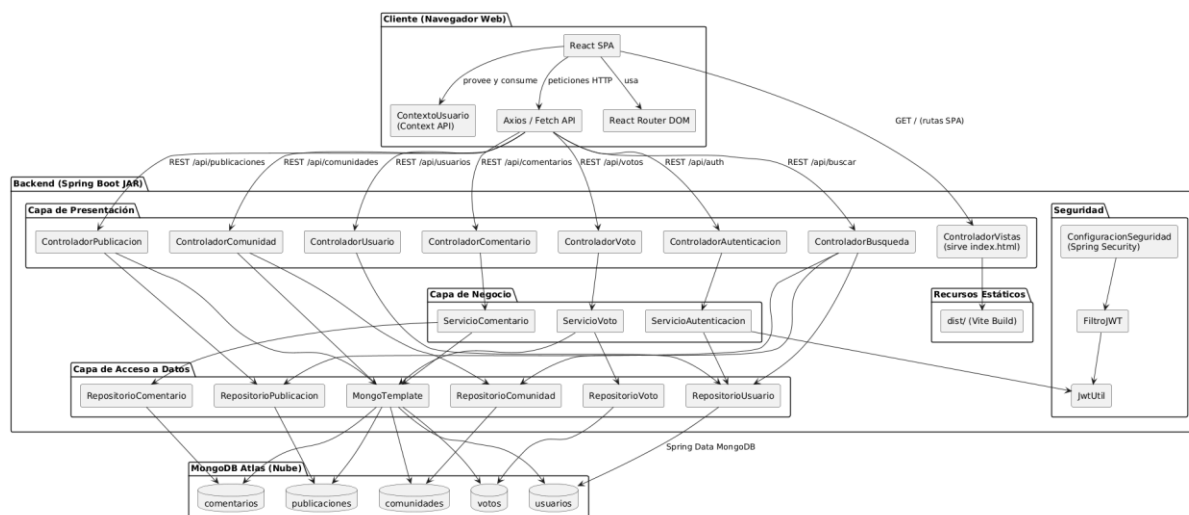


Ilustración 6. Diagrama de componentes

Descripción narrativa: El sistema se divide en tres grandes bloques: el cliente, el backend y la base de datos en la nube. El cliente (React SPA) se comunica con el backend exclusivamente mediante peticiones HTTP REST a través de Fetch/Axios, enviando automáticamente la cookie JWT en cada petición. El backend se estructura en cuatro capas internas: Presentación (controladores por recurso), Negocio (servicios especializados), Acceso a Datos (repositorios Spring Data y MongoTemplate) y Seguridad (filtro JWT, utilidad de token y configuración de Spring Security). El componente **ControladorVistas** sirve el **index.html** generado por Vite para cualquier ruta no reconocida como API, permitiendo que React Router maneje la navegación del lado del cliente. Finalmente, todos los repositorios y MongoTemplate se conectan a las colecciones correspondientes en MongoDB Atlas.

4. Implementación

4.1 Descripción general de la aplicación móvil

La aplicación web Ágora se compone de un backend desarrollado en Java con Spring Boot, mientras que el frontend fue construido con React 19 y Vite 8. Ambas partes del proyecto son independientes, la manera en la que operan los servidores separados es mediante comunicación a través de CORS, pese a ello, en producción ambos sirven como un único artefacto ejecutable (JAR).

4.2 Backend - Spring Boot

4.2.1 Tecnologías y dependencias principales

Ya entrando a detalles con respecto al backend de Ágora, este se desarrolló en **Java 21**, utilizando a **Spring Boot 3.x** como su framework principal. La gestión de dependencias se realizó con **Maven**, aquellas más esenciales para la realización del proyecto fueron:

Librería / Dependencia	Función en el proyecto
spring-boot-starter-web	Es el servidor HTTP embebido (Tomcat) el cual cuenta con soporte REST
spring-boot-starter-data-mongodb	Es la integración con MongoDB a través de MongoRepository y MongoTemplate
spring-boot-starter-security	La infraestructura de tanto la seguridad como filtros HTTP
jjwt-api / jjwt-impl / jjwt-jackson	Aportan la generación y validación de los JSON WEB Tokens (JWT)
BCrypt	Hashing de contraseñas gracias al algoritmo bcrypt
spring-boot-starter-validation	Aporta validación de entrada por anotaciones de Jakarta Validation
lombok	Ahorra la escritura del código repetitivo (getters, setters, constructores)

Tabla 6. Librerías y dependencias (con sus funcionalidades) utilizadas en el backend

El servidor se accede mediante el puerto **9090** sobre todas las interfaces de red (0.0.0.0), éstas se configuraron mediante variables de entorno.

4.2.2 Estructura de paquetes

El proyecto en backend sigue una organización en paquetes la cual refleja la arquitectura en capas:

src/main/java/com/agora/

```

├── controladores/  ← Los controladores REST forman la capa de presentación.
├── servicios/      ← Los servicios conforman la lógica del negocio.
├── repositorios/   ← Esta es la capa de datos, se declaran las interfaces de
                    MongoRepository.
├── modelos/        ← Las clases de dominio mapeadas a documentos de MongoDB
├── seguridad/      ← Aquí se encuentran el filtro JWT, la configuración CORS y Spring
                    Security.
└── utilidades/     ← JwtUtil así como helpers generales.
```

4.2.3 Controladores REST

Cada uno de los controladores agrupa sus respectivos endpoints de un recurso del dominio y fueron anotados con **@RestController**, dichos endpoints devuelven sus respuestas en formato JSON. Se procederá a detallar cada controlador implementado junto a su propósito:

ControladorAutenticacion: Es aquel que gestiona el ciclo de vida de la sesión del usuario, esto lo logra por medio de tres endpoints: «POST /api/auth/registro» para la creación de una nueva cuenta, «POST /api/auth/login» para su autenticación y posterior emisión de una cookie con el JWT, y «POST /api/auth/logout» para invalidar una sesión y eliminar así la cookie.

Adicionalmente, «GET /api/auth/verificar» es lo que permite al frontend validar la sesión activa y que se obtengan los datos del usuario autenticado.

ControladorUsuario: Se encarga de la consulta y actualización de perfiles de usuarios. Los endpoints «GET /api/usuarios/{id}» y «GET /api/usuarios/nombre/{nombreUsuario}» son los que obtienen los perfiles públicos, mientras que «PUT/api/usuarios/{id}» permite que tanto usuario como administrador pueda actualizar su foto de perfil y biografía. Por último, «GET /api/usuarios/{id}/publicaciones» es la que retoma las publicaciones de un usuario concreto, cuenta con soporte de paginación.

ControladorPublicacion: Hace gestión de la creación, consulta, edición y eliminación de cada publicación, su endpoint «GET /api/publicaciones» acepta dos filtros como parámetros (comunidadId, autorId), uno de ordenamiento (criterio: fecha o puntaje) y dos para la paginación (página, límite, con un máximo de 100 registros). «POST /api/publicaciones» es para la creación de nuevas publicaciones. «GET /api/publicaciones/{id}» es para retornar una publicación en concreto, mientras que «PUT /api/publicaciones/{id}» y «DELETE /api/publicaciones/{id}» son las que permiten su edición y eliminación. Los endpoints «PATCH /api/publicaciones/{id}/fijar» y «PATCH /api/publicaciones/{id}/bloquear» solo pueden ser accedidos por moderadores y administradores.

ControladorComentario: Se hace cargo de manejar los comentarios de cada publicación. «POST /api/comentarios» es quien crea un nuevo comentario o respuesta, recibe publicacionId, contenido y de manera opcional, padreId, en caso de ser parte de un hilo. «GET /api/comentarios/{publicacionId}» retorna todos los comentarios de una publicación concreta de manera ordenada para la reconstrucción de un árbol. «PUT /api/comentarios/{id}» y «DELETE /api/comentarios/{id}» permiten la edición y eliminación de un comentario, respectivamente.

ControladorVoto: Procesa los votos tanto de las publicaciones como los de comentarios. Su único endpoint, «POST /api/votos», recibe el un objetivoId, tipoObjetivo y valor, para delegar al ServicioVoto toda la lógica de inserción, retiro o cambio de voto. «GET /api/votos/estado» es lo que permite consultar si cada usuario ya votó algún elemento específico.

ControladorBusqueda: Da uso de «GET /api/buscar?q={termino}» para retornar resultados agrupados en tres listas: comunidades, publicaciones y usuarios que contengan la terminación buscada, dicha búsqueda es insensible a diferenciar mayúsculas o minúsculas de expresiones regulares de MongoDB.

4.2.4 Capa de seguridad

Las medidas de seguridad fueron implementadas con **Spring Security**, el cual se tuvo que adaptar a una arquitectura REST sin estado (stateless). Sus componentes clave son:

FiltroJWT: Intercepta cada petición HTTP antes de enviarlos al controlador, por lo mismo, es un filtrado del tipo OncePerRequestFilter. La manera en la que funciona es que extrae el JWT de la cookie token, pasa a validarlo con TwUtil.validarToken() y, de esta ser exitoso, se construye un objeto UsernamePasswordAuthenticationToken con los datos del usuario (id, nombre, usuario, rol) y es establecido en el SecurityContextHolder. En el caso de que el token resulte inválido o se tenga su ausencia, la petición continúa sin contexto de autenticación, dando de resultado un error 401 si el endpoint lo requiere.

JwtUtil: Es una clase que hace de utilidad por encapsular la generación y validación de tokens. Da uso de la librería jjwt con el algoritmo de firma HS256 (HMAC-SHA256). La variable de entorno JWT_SECRET es la que contiene la incógnita que se lee. Los tokens incluyen como claims el userId, nombreUsuario, email y rol de la respectiva clase Usuario, teniendo una expiración de 8 horas.

Configuración CORS: Se trata de un CorsConfigurationSource que permite el origen del frontend de desarrollo (http://localhost:5173), habilitar credenciales (allowCredentials: true) y autoriza los métodos HTTP GET, POST, PUT, DELETE, OPTIONS Y PATCH, así como los encabezados necesarios.

Configuración CSRF: Esta protección se deshabilita de manera explícita en la cadena de filtros de Spring Security, se hace debido a que la autenticación basada en cookies HttpOnly gestionadas por el servidor no son susceptibles a ataques CSRF, es decir, en el diseño actual el único cliente autorizado por sesión es la propia paginación que visualiza.

4.2.5 Servicios de negocio

ServicioAutenticacion: Aborda el registro de nuevos usuarios y el inicio de sesión. En el caso del registro, verifica la unicidad del correo electrónico y del nombre de usuario aplicando BCrypt.hashpw() a la contraseña antes de ser persistida, también asigna los valores iniciales para el karma, rol y fecha de creación. En cuanto al login, recupera a cada usuario mediante su email, comparando la contraseña introducida con el hash almacenado por medio de BCrypt.checkpw(), de validarse correctamente, invoca a JwtUtil.generarToken() para obtener el token a incluir en la cookie de respuesta.

ServicioComentario: Es la implementación lógica del patrón *Materialized Path*. Por cada comentario raíz, se iniciara un hilo como array vacío con su padreId como null. Por cada respuesta, se recupera el comentario padre, copiando su hilo y extendiendolo con el _id del padre al final del array, también valida que la profundidad no supere los 10 niveles. Luego de persistir el comentario, se incrementa el total_comentarios en la publicación correspondiente usando MongoTemplates con \$inc. En cuanto a las respuestas de la API, aquellos comentarios con eliminado = true verán reemplazado su contenido por un «[comentario eliminado]» con el autor anulado.

ServicioVoto: Es el más complejo de los tres por la lógica condicional implementada. La manera en cómo funciona es que mediante un voto entrante (userId, objectId, valor), el servicio de consulta a la colección VOTO, buscando un documento que coincida con esa combinación, de no haber voto previo, se inserta el nuevo voto y aplica \$inc sobre puntaje_votos (y votos_positivos / votos_negativos, en caso de ser publicación) del elemento votado, así como el karma del autor del contenido. Cuando exista un voto con valor distinto (osease, un cambio de voto), se actualiza el campo valor del documento VOTO y aplica el delta correcto sobre los contadores (+2 p -2, según la dirección del cambio).

4.2.6 Instrucciones NoSQL utilizadas en MongoDB

Algunas de las operaciones más relevantes realizadas sobre MongoDB, más allá de su CRUD estándar de Spring Data, fueron los siguientes:

\$inc (incremento atómico): Fue utilizado extensamente en ServicioVoto y ServicioComentario para la actualización de contadores sin necesidad de leer el documento por completo el documento:

```
mongoTemplate.updateFirst(  
    Query.query(Criteria.where("_id").is(publicacionId)),  
    new Update().inc("puntaje_votos", delta).inc("votos_positivos", deltaPositivos),  
    Publicacion.class  
);
```

Expresiones regulares para búsqueda: En el ControladorBusqueda, se utilizaron patrones de expresión regular de MongoDB a través de Criteria.where().regex() para la búsqueda insensible de mayúsculas y minúsculas.

```
Criteria.where("nombre").regex(termino, "i")
```

Consultas compuestas con múltiples condiciones: En ControladorVoto, la consulta del voto previo usa una combinación de tres criterios:

```
Query.query(Criteria.where("usuario_id").is(usuarioid)  
    .and("objetivo_id").is(objetivoid)  
    .and("tipo_objetivo").is(tipoObjetivo))
```

\$push / \$pull para arrays: Son los encargados de gestionar suscripciones a comunidades y arrays moderados, se tratan de instrucciones como \$addToSet para evitar duplicados y \$pull para eliminar el elemento sobre los arrays comunidades_suscritas y moderadores.

\$set para actualizaciones parciales: Cuando se cita un perfil o una publicación, se utiliza Update.set() para actualizar exclusivamente aquellos campos modificados, sin necesidad de reemplazar el documento completo.

4.3 Frontend - React con Vite

El fronted del proyecto fue desarrollado con React 19 como la biblioteca de interfaz de usuario, mientras que Vite 8 fue la herramienta de construcción y servidor de desarrollo. La gestión de dependencias se realizó con npm, algunas de las principales son:

Librería / Dependencias	Función en el proyecto
react / react-dom	Son tanto una biblioteca de UI como de renderizado DOM
vite	El bundler y servidor de desarrollo que incluye HMR
react-router-dom v7	Es el enrutamiento por el lado del cliente

	(SPA)
axios	Un cliente HTTP el cual consume las API REST
react-icons	La librería de iconos (flechas de votación, icono de UI) usada para los assets
react-hot-toast	Son notificaciones emergentes que dan retroalimentación al usuario

Tabla 7. Librerías y dependencias (con sus funcionalidades) utilizadas en el frontend

4.3.1 Tecnologías y dependencias principales

El frontend de Ágora está desarrollado con **React 19** como biblioteca de interfaz de usuario y **Vite 8** como herramienta de construcción y servidor de desarrollo. El proyecto es de tipo módulo ES ("**type**": "**module**") y no utiliza ninguna biblioteca de componentes externa ni de estilos de terceros; toda la interfaz se construye con CSS propio definido en **styles.css**.

Las dependencias de producción son mínimas e intencionalmente reducidas:

- **react** ^19.2.6 — biblioteca principal de UI.
- **react-dom** ^19.2.6 — renderizado en el DOM.
- **react-router-dom** ^7.16.0 — enrutamiento del lado del cliente (SPA).

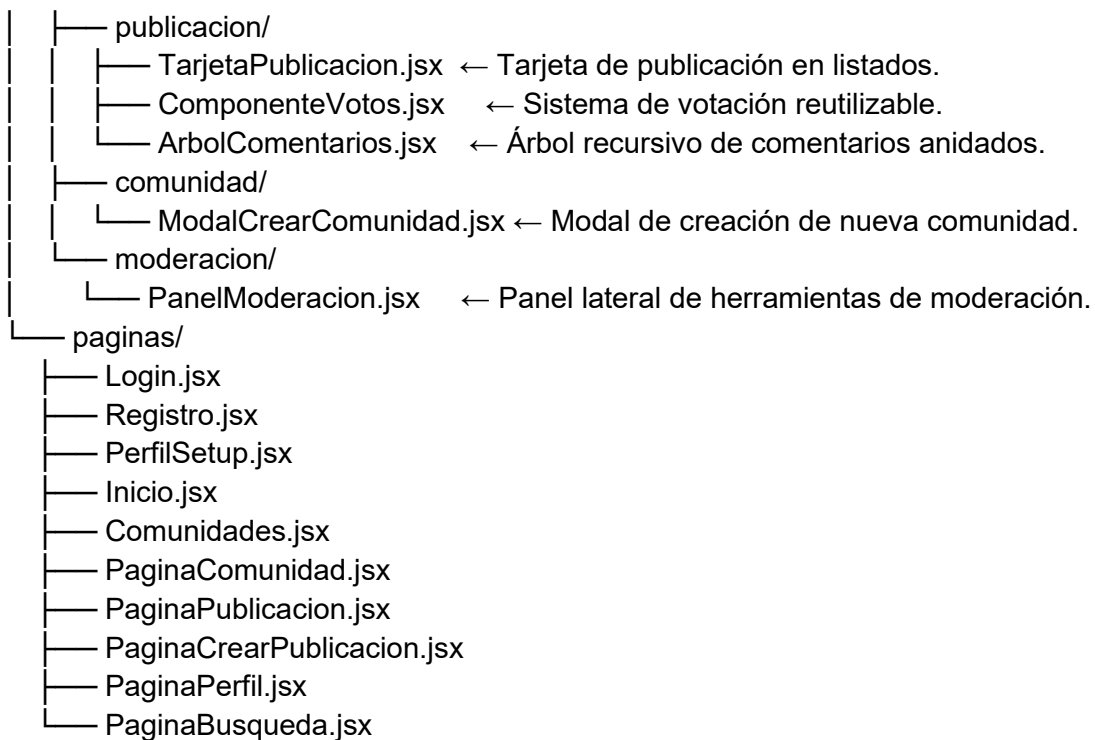
Las dependencias de desarrollo incluyen el plugin oficial **@vitejs/plugin-react** para soporte de JSX y Fast Refresh, ESLint con los plugins **eslint-plugin-react-hooks** y **eslint-plugin-react-refresh** para análisis estático.

4.3.2 Estructura de directorios

```

frontend/
├── index.html           ← Punto de entrada HTML (monta #root).
├── vite.config.js       ← Configuración de Vite y proxy de desarrollo.
├── package.json
├── eslint.config.js
├── public/
│   ├── imagenes/       ← Assets estáticos (logos, iconos de UI).
│   └── src/
│       ├── main.jsx     ← Punto de entrada React (monta VerificaAutenticacion + App).
│       ├── App.jsx      ← Definición del árbol de rutas.
│       ├── styles.css   ← Estilos globales de la aplicación.
│       ├── App.css
│       ├── contexto/
│       │   └── ContextoUsuario.jsx ← Contexto global de sesión y autenticación.
│       ├── componentes/
│       │   ├── RutaProtegida.jsx ← HOC que bloquea acceso sin sesión.
│       │   ├── RutaPublica.jsx  ← HOC que redirige si ya hay sesión activa.
│       │   └── layout/
│       │       ├── LayoutPrincipal.jsx ← Shell de página con navbar y área de contenido.
│       │       └── BarraBusqueda.jsx  ← Navbar global con búsqueda y menú de usuario.

```



4.3.3 Enrutamiento y protección de rutas

El enrutamiento se gestiona mediante **React Router DOM v7** con **BrowserRouter** en modo SPA. Todas las rutas se declaran en **App.jsx** y se dividen en dos categorías según su accesibilidad.

Rutas públicas — accesibles únicamente sin sesión activa. Si el usuario ya está autenticado, el componente **RutaPublica** redirige automáticamente a /:

Ruta	Componente
/login	Login
/registro	Registro
/perfil-setup	PerfilSetup

Tabla 8. Rutas públicas y sus componentes utilizados durante el enrutamiento

Rutas protegidas — requieren sesión activa. Si no hay sesión, **RutaProtegida** redirige a **/login** preservando la URL de origen en el estado de navegación para poder redirigir al usuario de vuelta tras el login. Todas las rutas protegidas se envuelven además en **LayoutPrincipal**:

Ruta	Componente
/	Inicio
/comunidades	Comunidades

/c/:id	PaginaComunidad
/p/:id	PaginaPublicacion
/publicar	PaginaCrearPublicacion
/u/:nombreUsuario	PaginaPerfil
/buscar	PaginaBusqueda

Tabla 9. Rutas privadas y sus componentes utilizados durante el enrutamiento

Cualquier ruta no reconocida (*) produce una redirección inmediata a /.

La lógica de protección funciona en tres estados posibles del contexto de usuario: **undefined** (verificación en curso, muestra pantalla de carga), **null** (sin sesión, redirige según tipo de ruta) y un objeto de usuario (sesión activa, permite el acceso).

4.3.4 Contexto de usuario

El estado de autenticación global se gestiona mediante la API de Context de React, implementada en **src/contexto/ContextoUsuario.jsx**. El contexto expone los siguientes valores y funciones a toda la aplicación:

Estado:

- **usuario**: objeto con los datos del usuario autenticado, **null** si no hay sesión, o **undefined** mientras se verifica.
- **cargando**: booleano que indica si la verificación inicial de sesión está en curso.

Funciones:

- **login(datosUsuario)**: actualiza el estado local con los datos del usuario recién autenticado (se llama desde las páginas de Login y Registro tras una respuesta exitosa).
- **logout()**: llama a **POST /api/auth/logout** para invalidar la cookie de sesión en el servidor y luego limpia el estado local.
- **verifica_sesion()**: función memoizada (**useCallback**) que realiza **GET /api/auth/yo** con **credentials: 'include'** para comprobar si existe una sesión válida. Se ejecuta automáticamente en el montaje inicial mediante **useEffect**.

Proveedor: VerificaAutenticacion es el componente proveedor que envuelve toda la aplicación en **main.jsx**. Al montarse, ejecuta **verifica_sesion()** para determinar si hay una cookie de sesión activa antes de renderizar cualquier ruta.

Hook de consumo: useAutentifica() es el hook personalizado que expone el contexto. Lanza un error explícito si se usa fuera del árbol de **VerificaAutenticacion**, garantizando un uso correcto.

4.3.5 Comunicación con la API

El frontend se comunica con el backend Spring Boot exclusivamente mediante **fetch** nativo del navegador. No se utiliza ninguna biblioteca HTTP externa como Axios.

Convenciones generales:

- Todas las peticiones incluyen **credentials: 'include'** para que el navegador envíe automáticamente la cookie de sesión JWT en cada solicitud.
- Las peticiones con cuerpo JSON incluyen la cabecera **'Content-Type': 'application/json'**.
- No existe una capa de servicio centralizada; cada componente o página realiza sus propias llamadas a la API directamente.

Proxy de desarrollo: Durante el desarrollo, Vite redirige todas las peticiones cuya URL comience por **/api** al servidor Spring Boot en **http://localhost:9090**, gracias a la configuración del proxy en **vite.config.js**. Esto permite que el frontend en el puerto 5173 y el backend en el 9090 operen sin problemas de CORS durante el desarrollo.

Optimistic updates (votos): El componente **ComponenteVotos** implementa actualización optimista: al votar, el estado local se actualiza inmediatamente antes de que el servidor responda. Si la respuesta del servidor indica error, el estado se revierte al valor previo para mantener la consistencia.

Método	Endpoint	Uso
GET	/api/auth/yo	Verificación de sesión activa
POST	/api/auth/logout	Cierre de sesión
GET	/api/usuarios/:id	Obtener datos de un usuario por ID
GET	/api/votos?objetivo=:id	Consultar voto actual del usuario
POST	/api/votos	Emitir o retirar un voto
POST	/api/comunidades	Crear una nueva comunidad
PUT	/api/comunidades/:id	Editar una comunidad existente
DELETE	/api/comunidades/:id	Eliminar una comunidad
GET	/api/comunidades/:id/miembros	Listar miembros de una comunidad
POST	/api/comunidades/:id/moderadores	Agregar moderador
DELETE	/api/comunidades/:id/moderadores/:id	Remover moderador

POST	/api/comunidades/:id/invitar	Invitar usuario a comunidad privada
DELETE	/api/comunidades/:id/miembros/:id	Expulsar miembro
POST	/api/publicaciones/:id/fijar	Fijar/desfijar publicación
POST	/api/publicaciones/:id/bloquear	Bloquear/desbloquear comentarios
DELETE	/api/publicaciones/:id	Eliminar publicación
DELETE	/api/comentarios/:id	Eliminar comentario (borrado suave)
GET	/api/buscar?q=:query	Búsqueda global

Tabla 10. Principales endpoints consumidos desde el frontend

4.3.6 Componentes principales

LayoutPrincipal

Shell de página que envuelve todas las rutas protegidas. Renderiza **BarraBusqueda** en la parte superior y debajo un contenedor de dos columnas: la columna principal (**<main>**) y, opcionalmente, una columna lateral (**<aside>**) si se pasa la prop **sidebar**. Cuando no hay sidebar, el área principal ocupa el ancho completo mediante clases CSS condicionales (**layout-main--con-sidebar / layout-main--completo**).

BarraBusqueda

Navbar global presente en todas las páginas de la aplicación. Contiene el logotipo de Ágora (enlazado a /), un formulario de búsqueda que navega a **/buscar?q=...** al enviarse, y el área de usuario. Cuando hay sesión, muestra el avatar del usuario (foto de perfil o iniciales) con un menú desplegable que da acceso al perfil, a la lista de comunidades y al cierre de sesión. El cierre del menú al hacer clic fuera se gestiona con un **ref** y un listener de **mousedown** sobre el documento.

TarjetaPublicacion

Componente de presentación que representa una publicación en cualquier listado. Recibe el objeto publicación, mapas de nombre de comunidad y autor por ID, y flags de moderación. Internamente mantiene el puntaje de votos en estado local (**puntaje_local**) para poder actualizarlo cuando **ComponenteVotos** informa de un cambio sin necesidad de recargar el listado completo. Soporta los tres tipos de publicación (texto, enlace, imagen) y muestra la miniatura de imagen si está disponible. Incluye un botón de eliminación que solo se renderiza cuando **es_moderador** y **al_eliminar** están presentes.

ComponenteVotos

Componente reutilizable de votación, usado tanto en publicaciones (orientación vertical) como en comentarios (orientación horizontal). Al montarse, consulta **GET /api/votos?objetivo=:id**

para cargar el voto existente del usuario y lo refleja visualmente en los botones. Al hacer clic, aplica actualización optimista y luego confirma o revierte según la respuesta del servidor. Formatea el puntaje de forma legible (por ejemplo, 1334 → "1.3k"). Deshabilita los botones mientras la carga inicial está en curso o durante el procesamiento de un voto, y también si no hay usuario autenticado.

ArbolComentarios

Componente que construye y renderiza el árbol de comentarios anidados de una publicación. Recibe el array plano de comentarios (que incluyen **padre_id** para representar la jerarquía) y construye internamente un mapa de hijos por **id** para organizar los nodos en árbol. Los comentarios raíz (sin **padre_id**) se muestran de forma paginada (5 por carga), con un botón "Cargar más comentarios" para incrementar la ventana visible. Cada nodo se representa con el componente interno **NodoComentario**, que maneja de forma recursiva la respuesta a comentarios, el colapso del hilo, el borrado suave con confirmación de dos pasos y la integración de **ComponenteVotos** en orientación horizontal.

ModalCrearComunidad

Modal de formulario para crear una nueva comunidad. Gestiona nombre (con validación de longitud 3–21 y expresión regular `^[a-zA-Z0-9_]+$`), descripción (hasta 500 caracteres), banner e icono (con selector dual: URL externa o archivo local que se convierte a base64 mediante **FileReader**, limitado a 5 MB), visibilidad (pública o privada) y un editor de reglas con hasta 15 entradas. Al enviarse navega automáticamente a la URL de la comunidad recién creada. Gestiona el foco inicial, el scroll del body y el cierre con la tecla Escape.

PanelModeracion

Panel lateral de administración de la comunidad, visible únicamente para moderadores y el administrador (creador). Muestra herramientas distintas según el contexto: en la vista de comunidad, permite editar la comunidad, agregar o remover moderadores, invitar usuarios (solo en comunidades privadas) y expulsar miembros. En la vista de publicación, permite fijar/desfijar la publicación y bloquear/desbloquear la sección de comentarios. Todas las acciones destructivas cuentan con un modal de confirmación. Las herramientas exclusivas del administrador (eliminar comunidad, remover moderadores) solo se muestran cuando **es_admin_comunidad** es **true**.

4.3.7 Construcción y despliegue

Desarrollo local

El proyecto se inicia con **npm run dev**, que arranca Vite en el puerto 5173 con Hot Module Replacement. El proxy configurado en **vite.config.js** redirige **/api/*** al backend Spring Boot en **localhost:9090**, por lo que ambos procesos deben ejecutarse simultáneamente durante el desarrollo. El servidor de desarrollo escucha en todas las interfaces (**host: '0.0.0.0'**), lo que permite el acceso desde otros dispositivos en la misma red.

Construcción para producción

El comando **npm run build** invoca **vite build**, que genera el bundle optimizado en **frontend/dist/**. Los assets compilados (un único archivo CSS y un único archivo JS con hashing de nombre, por ejemplo **index-0YDluccu.css** e **index-DEzuZ38u.js**) se copian manualmente al directorio **src/main/resources/dist/** del proyecto Spring Boot para que el backend los sirva como recursos estáticos.

Integración con el backend:

El controlador **ControladorVistas** en Spring Boot es responsable de servir el **index.html** del frontend para cualquier ruta que no sea un endpoint de la API. Esto implementa el patrón habitual de SPA donde el servidor siempre devuelve el HTML principal y React Router se encarga del enrutamiento del lado del cliente. Las imágenes estáticas de la UI (logos, iconos) también residen en **src/main/resources/dist/imagenes/** y se sirven directamente por el servidor.

5. Resultados

5.1 Autenticación y Perfil

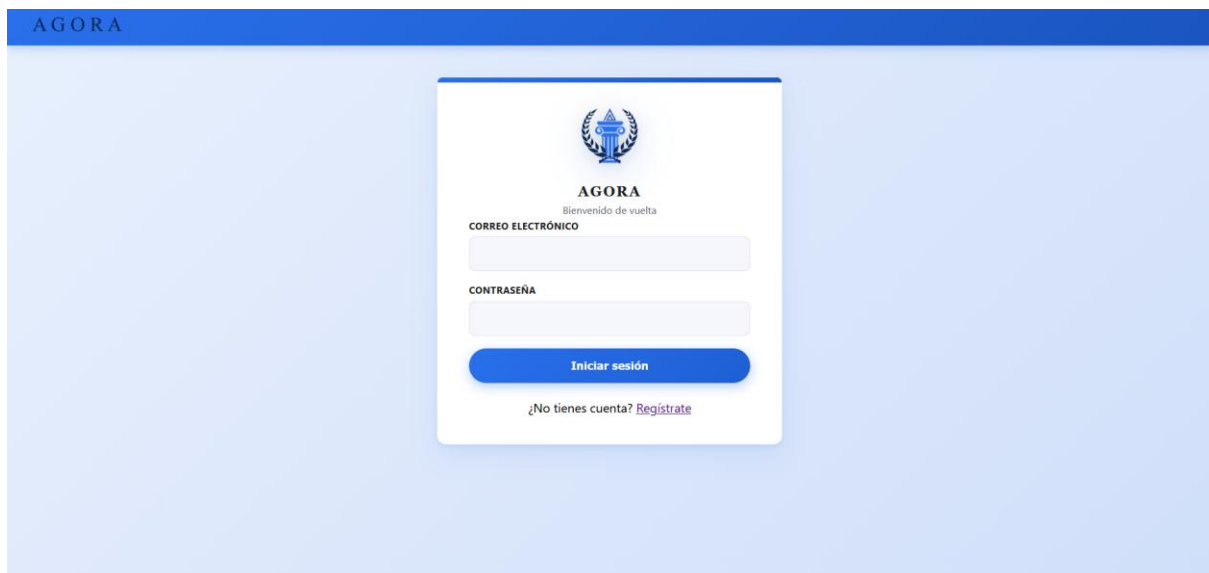


Ilustración 7. Pantalla de inicio de sesión

AGORA



AGORA
Crea tu cuenta

NOMBRE DE USUARIO

CORREO ELECTRÓNICO

CONTRASEÑA

CONFIRMAR CONTRASEÑA

[Continuar](#) -->

¿Ya tienes cuenta? [Inicia sesión](#)

Ilustración 8. Pantalla de registro

AGORA



AGORA
Personaliza tu perfil



Haz clic en la imagen para cambiar tu foto de perfil

BIOGRAFÍA

0 / 300

[Completar registro](#)

[Omitir por ahora](#)

Ilustración 9. Pantalla de configuración inicial del perfil

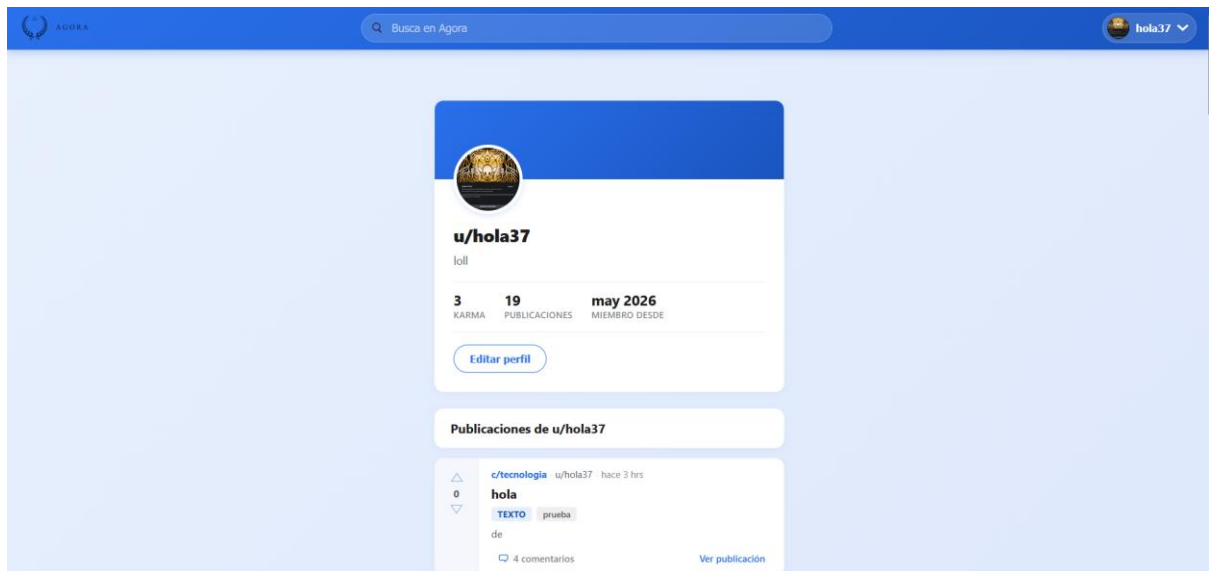


Ilustración 10. Página de perfil de usuario

5.2 Comunidades

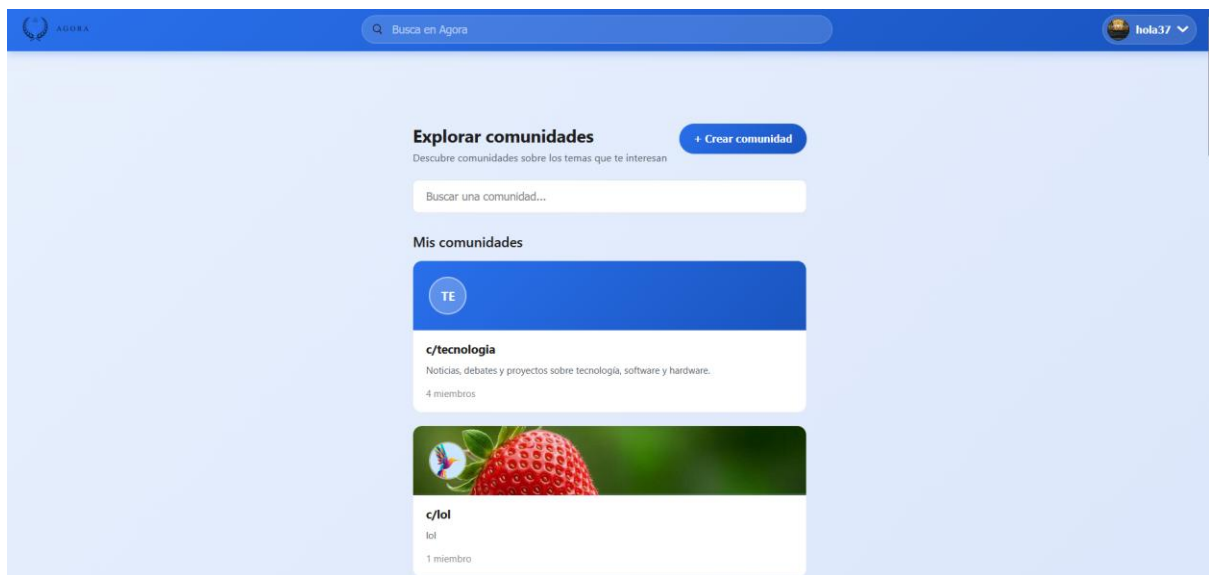


Ilustración 11. Listado de comunidades suscritas

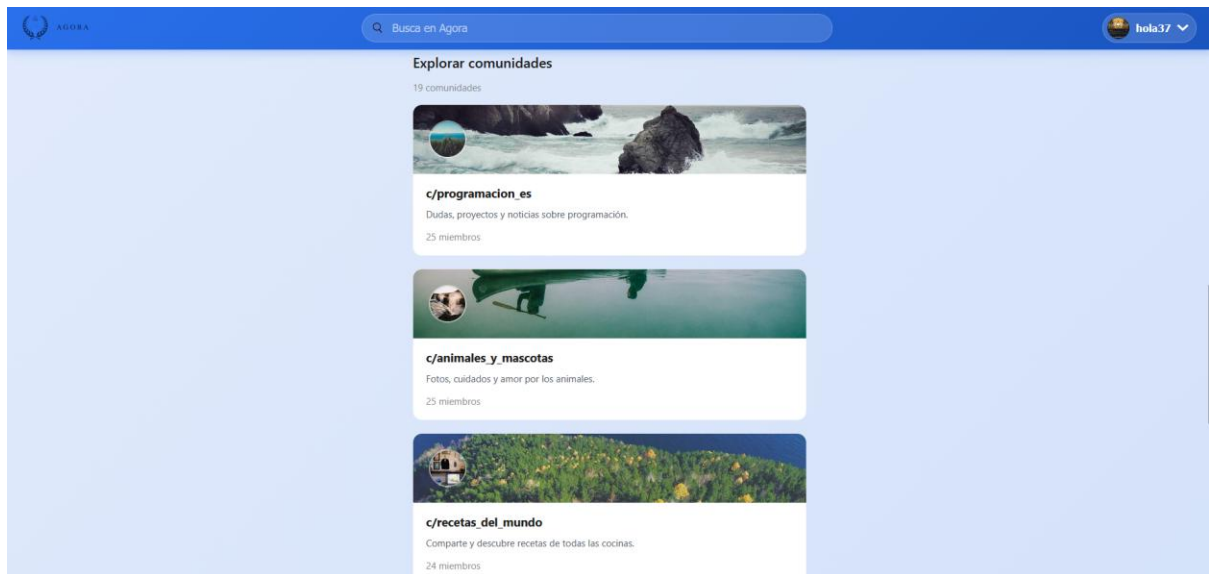


Ilustración 12. Listado de comunidades públicas

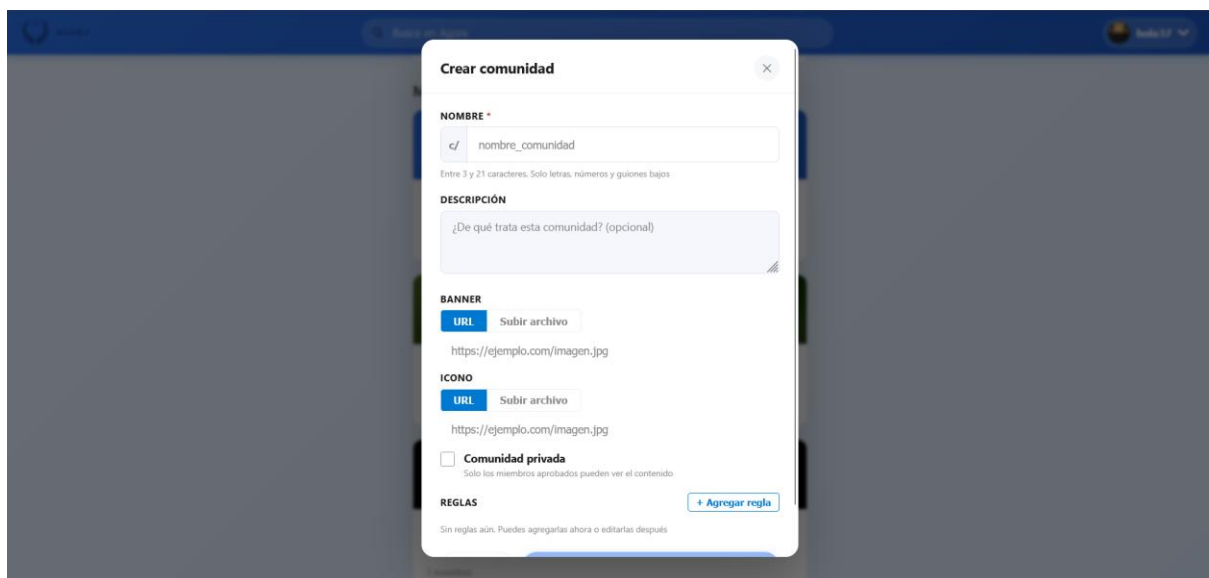


Ilustración 13. Modal de creación de comunidad

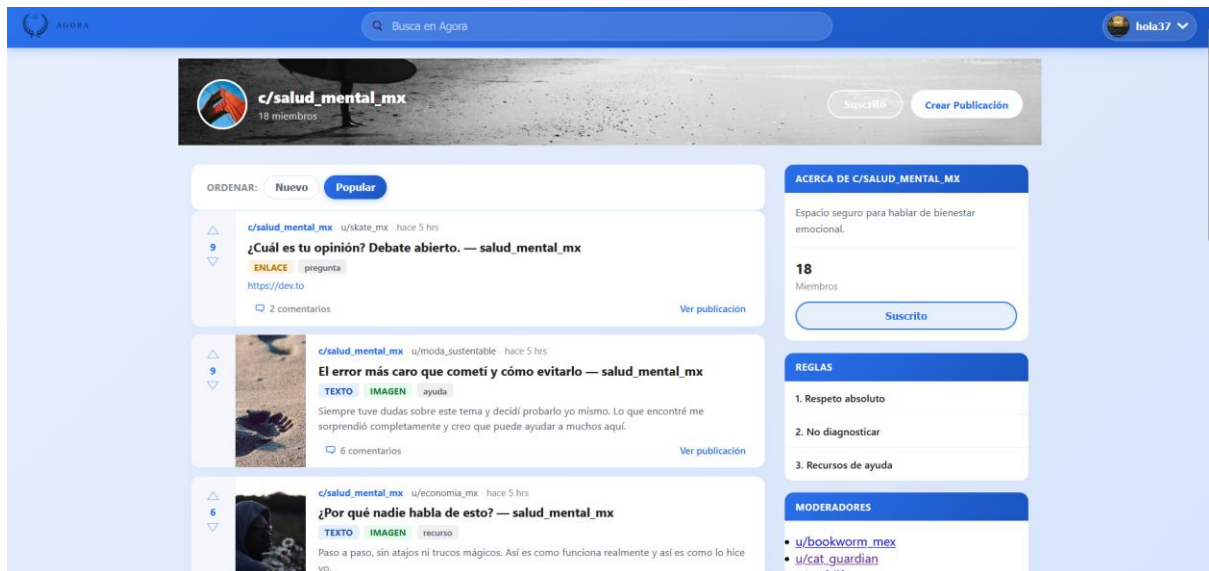


Ilustración 14. Página de una comunidad

5.3 Publicaciones

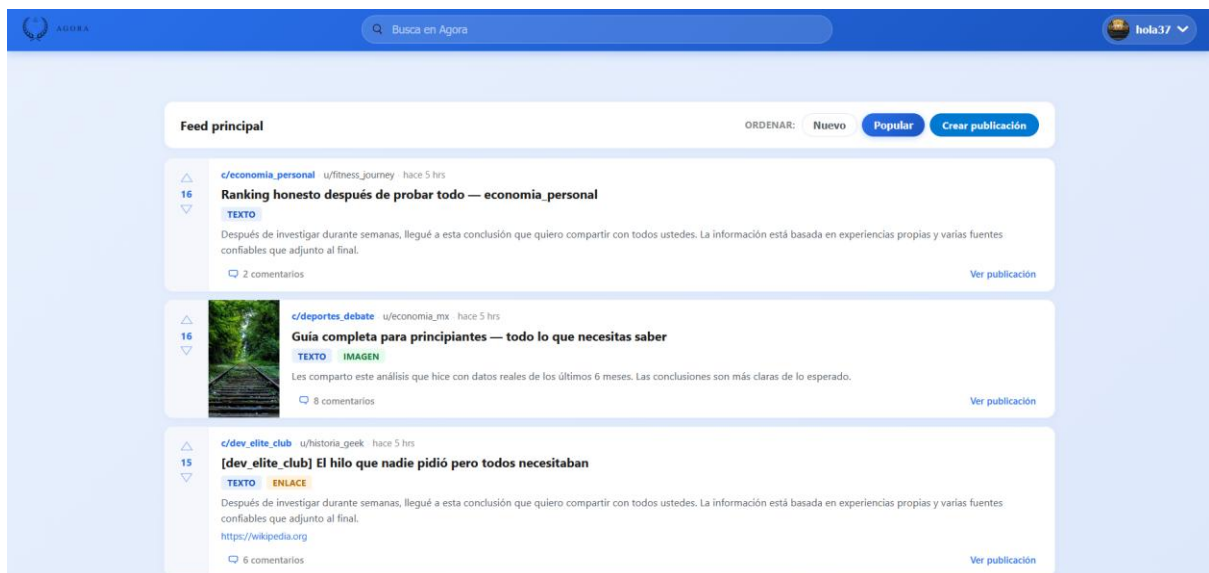


Ilustración 15. Feed de publicaciones (página de inicio)

 AGORA

Q Busca en Agora

 Hola37

Crear publicación

Volver al feed

Comunidad *

Buscar comunidad...

Título *

Un título descriptivo para tu publicación

0/300

Tipos de contenido (puedes combinar varios)

*Texto

*Enlace

*Imagen

Activa uno o más tipos. El contenido de cada tipo activo se guardará junto.

T Texto

Escribe el cuerpo de tu publicación (opcional)

E Enlace *

https://ejemplo.com/articulo

I Imagen *

URL

Subir archivo

Haz clic para seleccionar una imagen

Etiqueta (opcional)

ej. Pregunta, Noticia, Debate...

Clasifica tu publicación para que sea más fácil de encontrar

Cancelar

Publicar

TIPOS DE CONTENIDO

- **Texto** — cuerpo redactado libremente
- **Enlace** — URL a un recurso externo
- **Imagen** — URL directa a un archivo de imagen
- Puedes combinar los tres en una sola publicación

CONSEJOS

- Elige un título claro y descriptivo
- Verifica que la comunidad sea la adecuada
- Añade una etiqueta para facilitar la búsqueda

¿NO ENCUENTRAS TU COMUNIDAD?

Explora todas las comunidades disponibles o crea una nueva

[Ver comunidades](#)

Ilustración 16. Formulario de creación de publicación

38

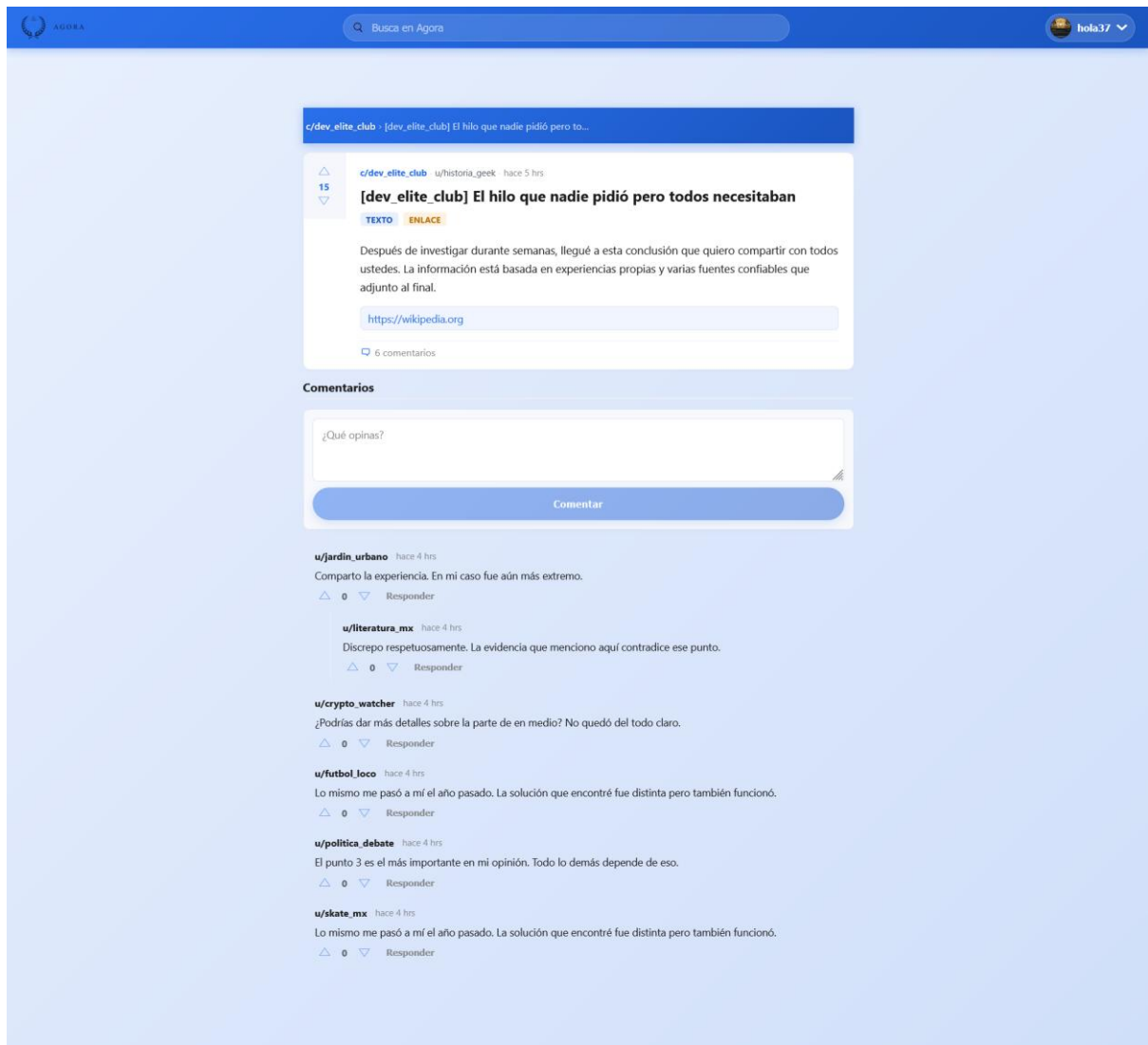


Ilustración 17. Página de detalle de una publicación de tipo texto

16

Guía completa para principiantes — todo lo que necesitas saber

TEXTO IMAGEN

A perspective view of a railway track receding into a dense forest, symbolizing a journey or path. The tracks are made of wooden sleepers and metal rails, leading the eye towards a bright opening in the trees in the distance. The surrounding foliage is lush and green, creating a sense of being deep within a woodland.

8 comentarios

¿Qué opinas?

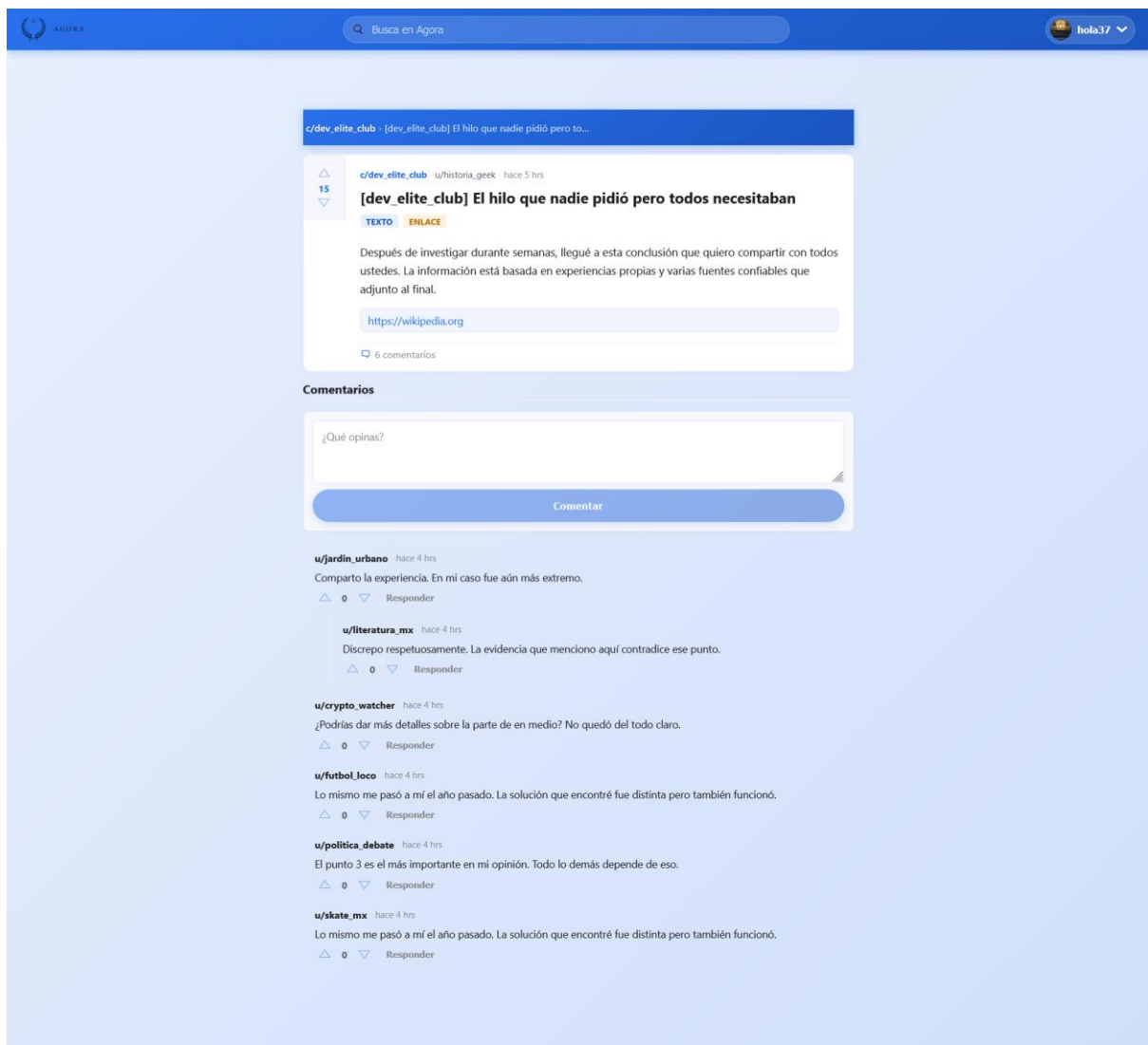


Ilustración 19. Página de detalles de una publicación de tipo enlace

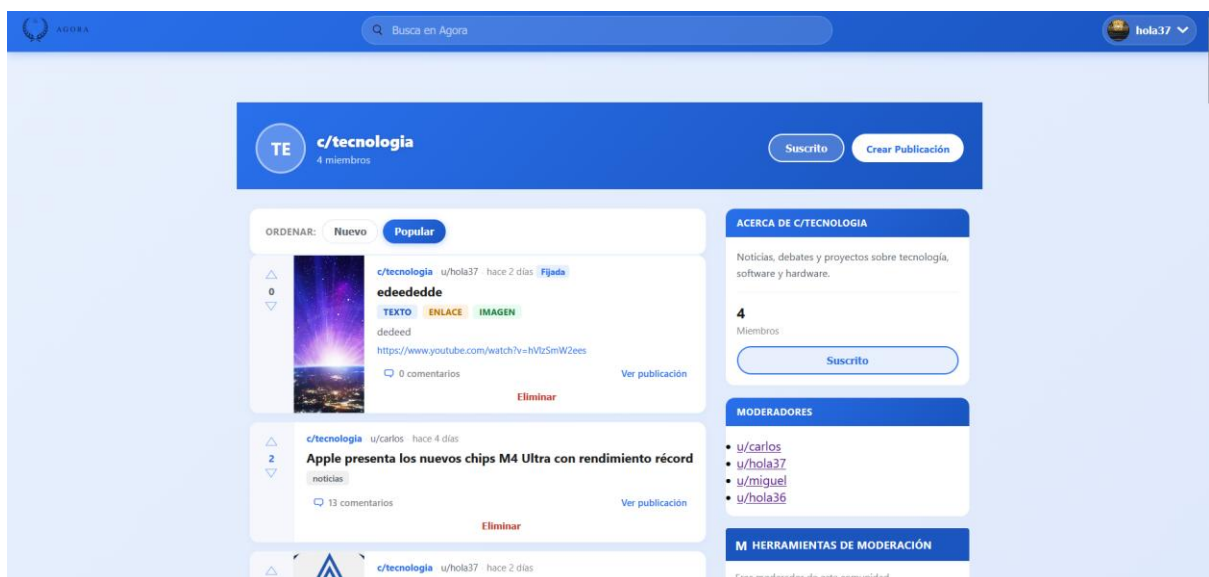


Ilustración 20. Publicación fijada

5.4 Comentarios y votos

The screenshot displays a forum thread with a light blue background. It features a series of nested comments, each with a user profile icon, username, and timestamp. The comments are as follows:

- u/laura** (hace 4 días): Impresionante. El salto de rendimiento en cargas de IA es real; lo probé con ComfyUI y la diferencia es abismal comparado al M3 Max. (2 upvotes, 1 downvote, 1 response, 1 delete)
- u/miguel** (hace 4 días): ¿Cuántos núcleos de Neural Engine tiene? El M3 tenía 16, me pregunto si duplicaron. (-1 upvote, 1 downvote, 1 response, 1 delete)
- u/laura** (hace 4 días): Tiene 38 núcleos de Neural Engine. Apple dice que procesa hasta 38 TOPS, casi el doble. (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/carlos** (hace 4 días): Exacto. Y lo más notable es que el consumo en reposo bajó, no subió. Muy eficiente. (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/miguel** (hace 4 días): Eso es lo que hace que sea tan interesante para laptops. Ahora sí llega al MacBook Air... (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/laura** (hace 4 días): En el Air sería brutal. Aunque el thermal throttling sin ventilador sigue siendo el talón de Aquiles. (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/carlos** (hace 4 días): También me interesa ver cómo se comporta en tareas de video 8K en Final Cut. Los benchmarks de DaVinci ya son increíbles. (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/miguel** (hace 4 días): ¿Alguien sabe cuándo llega a México y a qué precio? Apple siempre tarda siglos en actualizar el precio oficial en pesos. (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/laura** (hace 4 días): Normalmente aparece en la Apple Store MX a las 2-3 semanas del anuncio. El precio suele ser el dólar × 18 + IVA más un "margen Apple". (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/miguel** (hace 4 días): O sea que si está en \$3,999 USD, acá va a salir como a \$90k MXN. Sana economía 🤔 (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/carlos** (hace 4 días): Best Buy y iShop suelen recibirlos antes que apple.com/mx. Tienen los mismos precios pero con cashback con tarjeta BBVA a veces. (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/** (hace 4 días): [comentario eliminado] (0 upvotes, 1 downvote, 1 response, 1 delete)
- u/miguel** (hace 4 días): Respondo antes de que lo eliminen, para ver cómo queda el hilo con el padre eliminado. (0 upvotes, 1 downvote, 1 response, 1 delete)

Ilustración 21. Hilo de comentarios anidados

Basado en: Fig. 1

c/deportes_debate · u/economia_mx · hace 5 hrs

Guía completa para principiantes — todo lo que necesitas saber

TEXTO IMAGEN

Les comparto este análisis que hice con datos reales de los últimos 6 meses. Las conclusiones son más claras de lo esperado.



8 comentarios

Ilustración 22. Sistema de votos activos en una publicación

u/teatro_pasion hace 4 hrs
Este tema da para mucho más debate. Podría escribir páginas al respecto.
 **1**  **Responder**

u/surf_life hace 4 hrs
Lo mismo me pasó a mí el año pasado. La solución que encontré fue distinta pero también funcionó.
 **3**  **Responder**

u/idiomas_freak hace 4 hrs
Interesante perspectiva, aunque yo lo veo diferente. ¿Has considerado que...?
 **5**  **Responder**

u/recetas_abuela hace 4 hrs
Excelente publicación. La comparto con mi equipo ahora mismo.
 **0**  **Responder**

u/ciencia_popular hace 4 hrs
Llevas razón en todo menos en el último punto. Ahí creo que falta información.
 **-1**  **Responder**

u/cocinero_casero hace 4 hrs
El contexto importa mucho aquí. Para algunas personas no aplica igual.
 **5**  **Responder**

u/vegano_chef hace 4 hrs
¿Tienes fuentes para respaldar eso? No es que no te crea, solo quiero profundizar.
 **4**  **Responder**

Ilustración 23. Sistema de votos activos en comentarios

5.5 Búsqueda

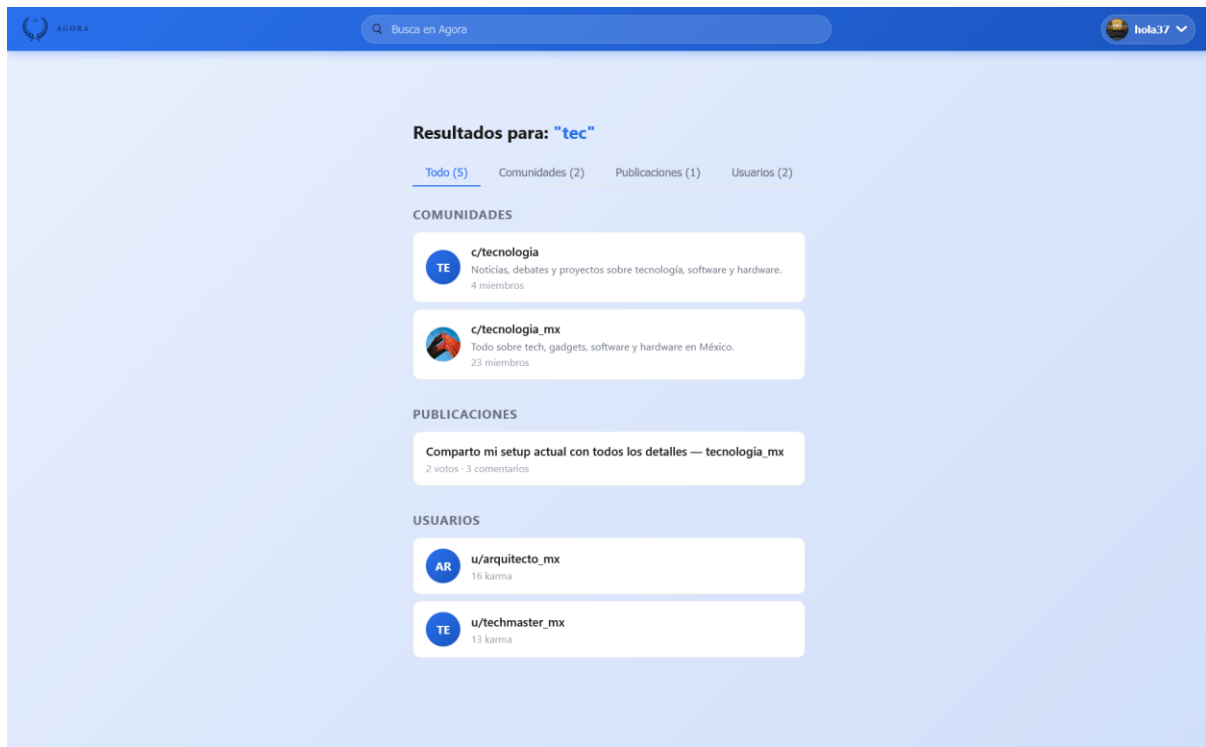


Ilustración 24. Página de resultados de una búsqueda completa

5.6 Panel de moderación



Ilustración 25. Panel de moderación



Ilustración 26. Panel de administración

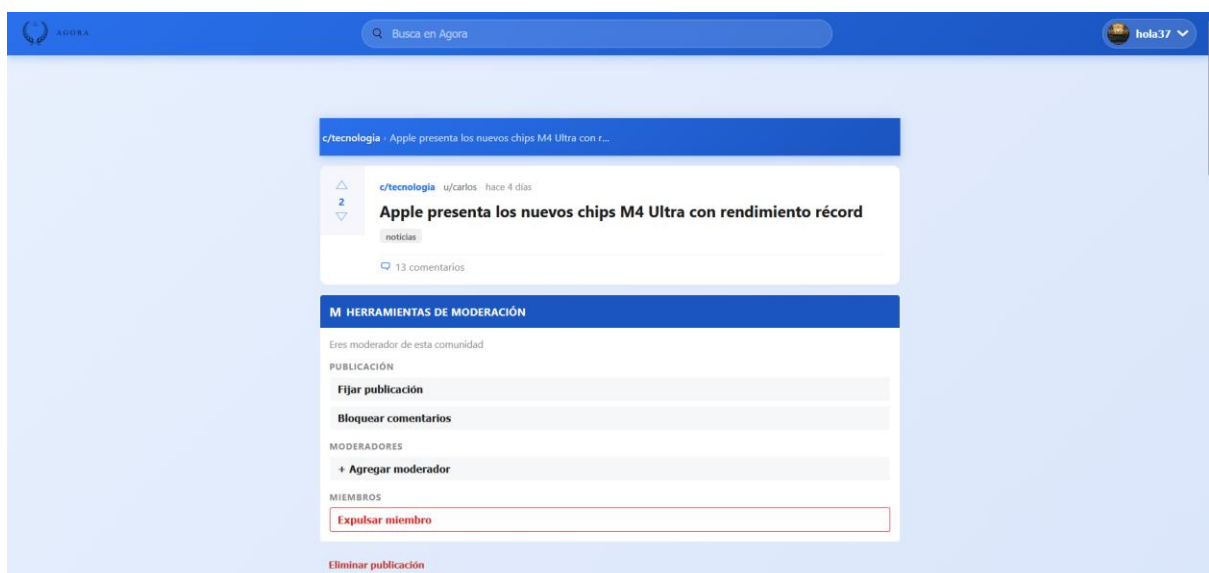


Ilustración 27. Panel de moderación de una publicación

6. Conclusiones

La presente sección expone los logros alcanzados durante el desarrollo de Ágora, así como las limitaciones identificadas tanto en la fase de diseño como durante la implementación. El análisis se realiza contrastando los objetivos planteados en la introducción con los resultados obtenidos.

6.1 Logros

Implementación completa del ciclo de vida de contenido. Se logró implementar de forma íntegra el flujo de creación, edición, eliminación y visualización de los tres tipos de contenido principales de la plataforma: publicaciones de texto, de enlace y de imagen. Este ciclo abarca desde la autenticación del autor hasta la renderización del contenido en el frontend, pasando por la validación del esquema en MongoDB Atlas, la lógica de negocio en el backend y la actualización reactiva de la interfaz en el cliente. La cobertura completa de este flujo representa el núcleo funcional del sistema y constituye el logro más representativo del proyecto.

Arquitectura cliente-servidor con separación efectiva de responsabilidades. Se implementó exitosamente una arquitectura de dos capas con separación total entre el frontend y el backend. El frontend, construido como una Single Page Application con React 19 y Vite, consume exclusivamente la API REST expuesta por el backend Spring Boot, sin acoplamiento directo con la base de datos ni con la lógica de negocio. Esta separación permitió que ambos módulos pudieran desarrollarse de forma paralela e independiente, lo que se tradujo en un flujo de trabajo más ágil dentro del equipo.

Sistema de autenticación seguro basado en JWT y cookies HttpOnly. Se implementó un mecanismo de autenticación robusto que utiliza JSON Web Tokens transmitidos mediante cookies HttpOnly, eliminando la exposición del token al entorno JavaScript del navegador. La gestión del ciclo de vida de la sesión (registro, inicio de sesión, verificación y cierre) funciona de manera coherente entre el frontend y el backend, y las rutas protegidas redirigen correctamente al usuario según su estado de autenticación.

Modelado NoSQL eficiente y validado a nivel de base de datos. El diseño de las cinco colecciones de MongoDB refleja decisiones de modelado fundamentadas en los patrones propios de las bases de datos orientadas a documentos. El uso de embedding para las reglas de comunidad, referencias por ObjectId para relaciones entre entidades independientes, contadores desnormalizados para puntajes y el patrón Materialized Path para los comentarios anidados demuestran una comprensión sólida del modelado NoSQL. Adicionalmente, los validadores JSON Schema configurados en MongoDB Atlas garantizan la integridad de los datos a nivel de la propia base de datos, independientemente de la capa de aplicación.

Sistema de votos con lógica de retiro y cambio, y actualización atómica de contadores. El módulo de votos implementa correctamente los tres estados posibles de interacción: emitir un voto nuevo, retirar un voto existente al repetirlo y cambiar el sentido de un voto previo. En los tres casos, los contadores de puntaje en la publicación o comentario votado y el karma del autor del contenido se actualizan mediante operaciones **\$inc** atómicas con

MongoTemplate, evitando condiciones de carrera y garantizando la consistencia de los datos incluso bajo carga concurrente.

Árbol de comentarios anidados con Materialized Path. Se implementó exitosamente el patrón Materialized Path para gestionar la jerarquía de comentarios anidados con hasta diez niveles de profundidad. Este enfoque permite recuperar la totalidad de los comentarios de una publicación en una única consulta a MongoDB y delegar al frontend la reconstrucción del árbol a partir de la longitud y contenido del campo **hilo**. La eliminación lógica de comentarios preserva la estructura del árbol aunque el contenido sea ocultado, evitando que los hilos de respuesta queden huérfanos.

Sistema de roles y moderación funcional. Se implementaron los tres niveles de acceso del sistema (usuario, moderador y administrador) con verificaciones de autorización en el backend para cada operación sensible. Los moderadores disponen de un panel de moderación operativo desde el cual pueden gestionar publicaciones (fijar, bloquear, eliminar), miembros (expulsar), moderadores (agregar o remover) e información de la comunidad (editar nombre, descripción, banner, ícono y reglas).

Motor de búsqueda global integrado. Se implementó un endpoint de búsqueda que retorna resultados agrupados en tres categorías (comunidades, publicaciones y usuarios) a partir de un único término de consulta, con soporte de búsqueda insensible a mayúsculas y minúsculas. El frontend presenta sugerencias en tiempo real mientras el usuario escribe y permite navegar a una página de resultados completa.

Soporte de comunidades privadas con sistema de invitaciones. El sistema soporta comunidades de tipo privado que no aparecen en el listado público. Los moderadores pueden invitar usuarios específicos mediante su identificador, y los usuarios invitados pueden aceptar o rechazar la invitación desde su interfaz, lo que añade una capa de control de acceso a nivel de comunidad más allá de los roles globales.

6.2 Limitaciones

Ausencia de notificaciones en tiempo real. El sistema no implementa ningún mecanismo de comunicación bidireccional en tiempo real, como WebSockets o Server-Sent Events. Los usuarios no reciben alertas inmediatas ante eventos relevantes, como respuestas a sus comentarios, votos recibidos o invitaciones a comunidades privadas. Las invitaciones pendientes solo son visibles si el usuario navega activamente a la sección correspondiente. Esta limitación impacta directamente en la experiencia de usuario en comparación con plataformas maduras como Reddit, donde las notificaciones son un elemento central de la interacción.

Almacenamiento de imágenes por URL externa, sin carga de archivos. El sistema no implementa un módulo de almacenamiento de archivos propio ni integración con servicios de terceros como Amazon S3, Cloudinary o Firebase Storage. Las imágenes de perfil, banners, íconos de comunidad y publicaciones de tipo imagen se referencian únicamente mediante URL externas proporcionadas por el usuario. Esto implica que si la URL de origen deja de

estar disponible, la imagen desaparece de la plataforma sin posibilidad de recuperación, lo que compromete la integridad visual del contenido a largo plazo.

Paginación limitada y ausencia de scroll infinito. Si bien el backend implementa paginación del lado del servidor mediante los parámetros **pagina** y **limite**, el frontend no implementa carga dinámica de publicaciones mediante scroll infinito. El usuario debe navegar manualmente entre páginas de resultados, lo que resulta en una experiencia menos fluida en comparación con aplicaciones modernas de contenido continuo.

Sin sistema de reportes de contenido. La plataforma no cuenta con un mecanismo que permita a los usuarios reportar publicaciones o comentarios con contenido inapropiado. Toda la moderación depende de la acción proactiva de los moderadores o del administrador global, lo que puede resultar insuficiente en comunidades con alto volumen de actividad. Esta ausencia representa una brecha significativa en los mecanismos de control de calidad del contenido generado por los usuarios.

Mensajería directa no implementada. No existe un módulo de mensajería privada entre usuarios. La comunicación dentro de la plataforma está restringida al ámbito público de las publicaciones y sus hilos de comentarios. Esto limita las posibilidades de interacción entre usuarios que deseen comunicarse de forma privada sin necesidad de exponer su conversación en una comunidad.

Sin despliegue en entorno de producción en la nube. El sistema fue desarrollado y probado en entorno local. No se realizó un despliegue en un servicio de nube pública (como AWS, Google Cloud o Railway), por lo que la aplicación no es accesible de forma pública desde internet. Si bien el uso de MongoDB Atlas como base de datos en la nube y la configuración mediante variables de entorno preparan el sistema para un despliegue futuro, este paso quedó fuera del alcance del proyecto académico.

Ausencia de pruebas automatizadas. El proyecto no incluye una suite de pruebas unitarias ni de integración automatizadas. La verificación del comportamiento del sistema se realizó de forma manual durante el desarrollo. Esto implica que modificaciones futuras en la lógica de negocio o en los controladores no cuentan con una red de seguridad que detecte regresiones, lo que aumenta el riesgo de introducir errores al extender el sistema.

7. Referencias

- [1] Pivotal Software, *Spring Boot Reference Documentation*, versión 3.x. Spring Projects, 2024. [En línea]. Disponible en: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [2] Pivotal Software, *Spring Data MongoDB Reference Documentation*. Spring Projects, 2024. [En línea]. Disponible en: <https://docs.spring.io/spring-data/mongodb/docs/current/reference/html/>
- [3] Pivotal Software, *Spring Security Reference Documentation*. Spring Projects, 2024. [En línea]. Disponible en: <https://docs.spring.io/spring-security/reference/>
- [4] MongoDB, Inc., *MongoDB Manual: Document Validation*. MongoDB, 2024. [En línea]. Disponible en: <https://www.mongodb.com/docs/manual/core/schema-validation/>
- [5] MongoDB, Inc., *MongoDB Manual: Model Tree Structures with Materialized Paths*. MongoDB, 2024. [En línea]. Disponible en: <https://www.mongodb.com/docs/manual/tutorial/model-tree-structures-with-materialized-paths/>
- [6] MongoDB, Inc., *MongoDB Atlas Documentation*. MongoDB, 2024. [En línea]. Disponible en: <https://www.mongodb.com/docs/atlas/>
- [7] MongoDB, Inc., *MongoDB Manual: Atomic Operations and Isolation*. MongoDB, 2024. [En línea]. Disponible en: <https://www.mongodb.com/docs/manual/core/write-operations-atomicity/>
- [8] Meta Platforms, Inc., *React Documentation: Main Concepts*, versión 19. React, 2024. [En línea]. Disponible en: <https://react.dev/>
- [9] Meta Platforms, Inc., *React Documentation: Context API*. React, 2024. [En línea]. Disponible en: <https://react.dev/reference/react/createContext>
- [10] Remix Software, *React Router DOM Documentation*, versión 7. React Router, 2024. [En línea]. Disponible en: <https://reactrouter.com/>
- [11] Vite Contributors, *Vite Documentation*, versión 8. Vite, 2024. [En línea]. Disponible en: <https://vitejs.dev/guide/>
- [12] M. Jones, J. Bradley y N. Sakimura, *JSON Web Token (JWT)*, RFC 7519. Internet Engineering Task Force (IETF), mayo 2015. [En línea]. Disponible en: <https://datatracker.ietf.org/doc/html/rfc7519>
- [13] N. Provos y D. Mazières, "A Future-Adaptable Password Scheme," en *Proceedings of the USENIX Annual Technical Conference*, Monterey, CA, EE. UU., 1999, pp. 81–91.
- [14] R. Fielding y J. Reschke, *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*, RFC 7231. Internet Engineering Task Force (IETF), junio 2014. [En línea]. Disponible en: <https://datatracker.ietf.org/doc/html/rfc7231>

- [15] R. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, tesis doctoral, University of California, Irvine, CA, EE. UU., 2000. [En línea]. Disponible en: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [16] W3C, *Cross-Origin Resource Sharing (CORS)*, W3C Recommendation. World Wide Web Consortium, enero 2014. [En línea]. Disponible en: <https://www.w3.org/TR/cors/>
- [17] IETF, *The OAuth 2.0 Authorization Framework: Bearer Token Usage*, RFC 6750. Internet Engineering Task Force (IETF), octubre 2012. [En línea]. Disponible en: <https://datatracker.ietf.org/doc/html/rfc6750>
- [18] K. Chodorow, *MongoDB: The Definitive Guide*, 3.^a ed. Sebastopol, CA, EE. UU.: O'Reilly Media, 2019.
- [19] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, EE. UU.: Addison-Wesley, 1994.
- [20] Oracle Corporation, *Java SE 17 Documentation*. Oracle, 2024. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/17/>
- [21] jBCrypt Contributors, *jBCrypt: A Java Implementation of OpenBSD's Blowfish Password Hashing*. [En línea]. Disponible en: <https://www.mindrot.org/projects/jBCrypt/>
- [22] ECMA International, *ECMAScript 2023 Language Specification*, ECMA-262, 14.^a ed. Ginebra, Suiza: ECMA International, junio 2023. [En línea]. Disponible en: <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>
- [23] Internet Engineering Task Force, *The Secure HyperText Transfer Protocol*, RFC 2818. IETF, mayo 2000. [En línea]. Disponible en: <https://datatracker.ietf.org/doc/html/rfc2818>